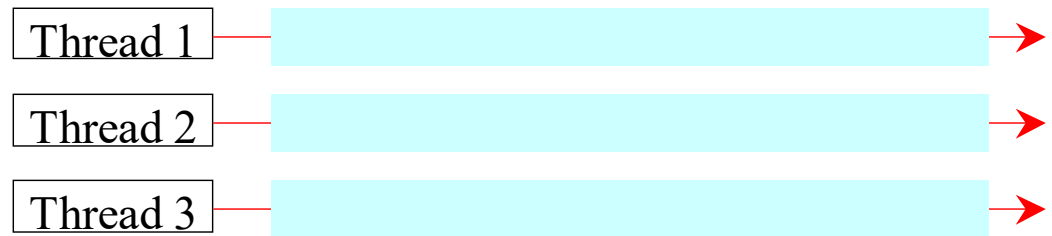


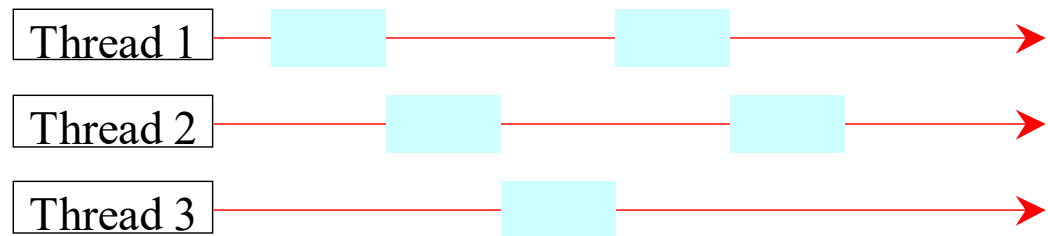
25 Multithreading

Threads Concept

Multiple
threads on
multiple
CPUs



Multiple
threads
sharing a
single CPU



Creating Tasks and Threads

`java.lang.Runnable`

TaskClass



```
// Custom task class
public class TaskClass implements Runnable {
    ...
    public TaskClass(...) {
        ...
    }

    // Implement the run method in Runnable
    public void run() {
        // Tell system how to run custom thread
        ...
    }
    ...
}
```

```
// Client class
public class Client {
    ...
    public void someMethod() {
        ...
        // Create an instance of TaskClass
        TaskClass task = new TaskClass(...);

        // Create a thread
        Thread thread = new Thread(task);

        // Start a thread
        thread.start();
        ...
    }
    ...
}
```

Example:

Using the `Runnable` Interface to Create and Launch Threads

- Objective: Create and run three threads:
 - The first thread prints the letter *a* 100 times.
 - The second thread prints the letter *b* 100 times.
 - The third thread prints the integers 1 through 100.

[TaskThreadDemo](#)

Run

```
22 class PrintChar implements Runnable {
23     private char charToPrint; // The character to print
24     private int times; // The times to repeat
25
26     /** Construct a task with specified character and number of
27      * times to print the character
28      */
29     public PrintChar(char c, int t) {
30         charToPrint = c;
31         times = t;
32     }
33
34     @Override /** Override the run() method to tell the system
35      * what the task to perform
36      */
37     public void run() {
38         for (int i = 0; i < times; i++) {
39             System.out.print(charToPrint);
40         }
41     }
42 }
```

```

44 // The task class for printing number from 1 to n for a given n
45 class PrintNum implements Runnable {
46     private int lastNum;
47
48     /** Construct a task for printing 1, 2, ... i */
49     public PrintNum(int n) {
50         lastNum = n;
51     }
52
53     @Override /** Tell the thread how to run */
54     public void run() {
55         for (int i = 1; i <= lastNum; i++) {
56             System.out.print(" " + i);
57             Thread.yield();
58         }
59     }
60 }

```

61

```

b
1baaaaaaaaaaabbbaaaabbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
baaaaaaaaaaaaaaaaaaaaaaaaaabbbbbbbbaaaaaaaaaa
aaaaaaabbbbbbbbbbbbaaaaaaaaaaaaaaaaaaaaaaaa
a 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41
42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79
80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98
99 100

```

```

public static void main(String[] args) {
    // Create tasks
    Runnable printA = new PrintChar('a', 100);
    Runnable printB = new PrintChar('b', 100);
    Runnable print100 = new PrintNum(100);

    // Create threads
    Thread thread1 = new Thread(printA);
    Thread thread2 = new Thread(printB);
    Thread thread3 = new Thread(print100);

    // Start threads
    thread1.start();
    thread2.start();
    thread3.start();
}

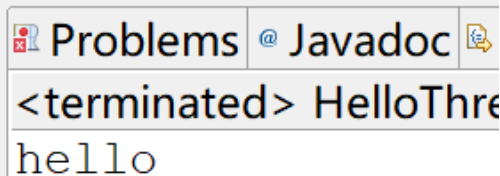
```

Creating Tasks and Threads

```
public class HelloThread extends Thread{  
    public void run() {  
        System.out.println("hello");  
    }  
    public static void main(String[] args) {  
        Thread thread = new HelloThread();  
        thread.start();  
    }  
}
```

```
<terminated> HelloT  
hello
```

```
public class HelloThread extends Thread{  
    public void run() {  
        System.out.println("hello");  
    }  
    public static void main(String[] args) {  
        Thread thread = new HelloThread();  
        thread.run();  
    }  
}
```



The screenshot shows the 'Problems' tab in an IDE, with a sub-tab for 'Javadoc'. It displays a message: '<terminated> HelloThre' followed by the output 'hello' on the next line.

区别？

Thread的一个静态方法，返回当前执行的线程对象：

```
public static native Thread currentThread();
```

每个Thread都有一个id和name:

```
public long getId();  
public final String getName();
```

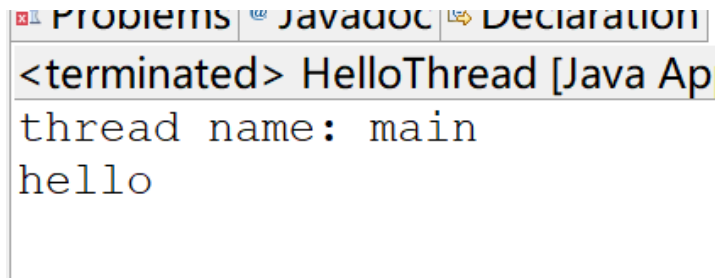
```
public class HelloThread extends Thread{  
    public void run() {  
        System.out.println("thread name: "  
            + Thread.currentThread().getName());  
        System.out.println("hello");  
    }  
    public static void main(String[] args) {  
        Thread thread = new HelloThread();  
        thread.start();  
    }  
}
```

```
<terminated> HelloThread [Java Applic  
thread name: Thread-0  
hello
```

```

public class HelloThread extends Thread{
    public void run() {
        System.out.println("thread name: "
            + Thread.currentThread().getName());
        System.out.println("hello");
    }
    public static void main(String[] args) {
        Thread thread = new HelloThread();
        thread.run();
    }
}

```



Problems Javadoc Declaration
 <terminated> HelloThread [Java Ap
 thread name: main
 hello

- 直接调用run，虽然也会运行run方法，但并不会启动一条单独的执行流，依然在main线程中执行。

The Thread Class

«interface»
java.lang.Runnable



java.lang.Thread	
+Thread()	Creates a default thread.
+Thread(task: <u>Runnable</u>)	Creates a thread for a specified task.
+start(): void	Starts the thread that causes the run() method to be invoked by the JVM.
+isAlive(): boolean	Tests whether the thread is currently running.
+setPriority(p: int): void	Sets priority p (ranging from 1 to 10) for this thread.
+join(): void	Waits for this thread to finish.
+ <u>sleep(millis: long): void</u>	Puts the runnable object to sleep for a specified time in milliseconds.
+ <u>yield(): void</u>	Causes this thread to temporarily pause and allow other threads to execute.
+interrupt(): void	Interrupts this thread.

The Static yield() Method

You can use the yield() method **to temporarily release time for other threads**. For example, suppose you modify the code in Lines 53-57 in TaskThreadDemo.java as follows:

```
public void run() {  
    for (int i = 1; i <= lastNum; i++) {  
        System.out.print(" " + i);  
        Thread.yield();  
    }  
}
```

Every time a number is printed, the print100 thread is yielded. So, the numbers are printed after the characters.

The Static sleep(milliseconds) Method

The sleep(long mills) method puts the thread to sleep for the specified time in milliseconds. For example, suppose you modify the code in Lines 53-57 in TaskThreadDemo.java as follows:

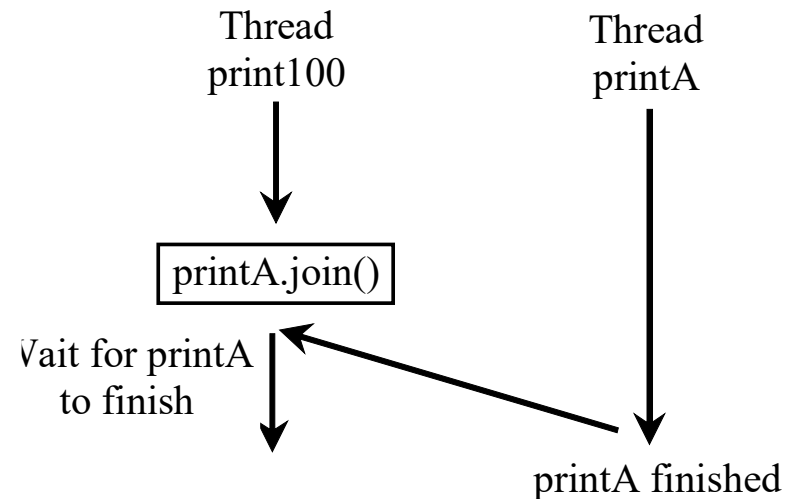
```
public void run() {  
    for (int i = 1; i <= lastNum; i++) {  
        System.out.print(" " + i);  
        try {  
            if (i >= 50) Thread.sleep(1);  
        }  
        catch (InterruptedException ex) {  
        }  
    }  
}
```

Every time a number (≥ 50) is printed, the print100 thread is put to sleep for 1 millisecond.

The join() Method

You can use the **join()** method to force one thread to wait for another thread to finish. For example, suppose you modify the code in Lines 53-57 in TaskThreadDemo.java as follows:

```
public void run() {  
    Thread threadA = new Thread(  
        new PrintChar('c', 40));  
    threadA.start();  
    try {  
        for (int i = 1; i <= lastNum; i++) {  
            System.out.print(" " + i);  
            if (i == 50) threadA.join();  
        }  
    }  
    catch (InterruptedException ex) {  
    }  
}
```



The numbers after 50 are printed after thread `printA` is finished.

isAlive(), interrupt(), and isInterrupted()

The **isAlive()** method is used to find out the state of a thread. It returns **true** if a thread is in the **Ready, Blocked, or Running state**; it returns **false** if a thread is **new and has not started** or if it is finished.

The **interrupt()** method interrupts a thread in the following way: **If a thread is currently in the Ready or Running state**, its interrupted flag is set; **if a thread is currently blocked**, it is **awakened** and enters the Ready state, and an `java.io.InterruptedException` is thrown.

The **isInterrupted()** method tests whether the thread is interrupted.

- `interrupt()` 方法只是改变中断状态而已，它不会中断一个正在运行的线程。
- 这一方法实际完成的是，给受阻塞的线程发出一个中断信号，这样受阻线程就得以退出阻塞的状态。
- 更确切的说，如果线程被`Object.wait`, `Thread.join`和`Thread.sleep`三种方法之一阻塞，此时调用该线程的`interrupt()`方法，那么该线程将抛出一个`InterruptedException`中断异常（该线程必须事先预备好处理此异常），从而提早地终结被阻塞状态。
- 如果线程没有被阻塞，这时调用 `interrupt()`将不起作用，直到执行到`wait()`,`sleep()`,`join()`时,才马上会抛出 `InterruptedException`。

- 当线程由于被调用了sleep(), wait(), join()等方法而进入阻塞状态；若此时调用线程的interrupt()将线程的中断标记设为true。由于处于阻塞状态，中断标记会被清除，同时产生一个InterruptedException异常。将InterruptedException放在适当的位置就能终止线程

```
@Override
public void run() {
    try {
        while (true) {
            // 执行任务...
        }
    } catch (InterruptedException ie) {
        // 由于产生InterruptedException异常，退出while(true)循环，线程终止！
    }
}
```

注意：对InterruptedException的捕获一般放在while(true)循环体的外面，这样，在产生异常时就退出了while(true)循环。否则，InterruptedException在while(true)循环体内，就需要额外的添加退出处理。

```
public void run() {
    while (true) {
        try {
            // 执行任务...
        } catch (InterruptedException ie) {
            // InterruptedException在while(true)循环体内。
            // 当线程产生了InterruptedException异常时，while(true)仍能继续运行！需要手动退出
            break;
        }
    }
}
```

```
class MyThread extends Thread {
```

```
    public MyThread(String name) {  
        super(name);  
    }  
  
    @Override  
    public void run() {  
        try {  
            int i=0;  
            while (!isInterrupted()) {  
                Thread.sleep(100); // 休眠100ms  
                i++;  
                System.out.println(Thread.currentThread().getName()+" (" +this.getState()+") loop " + i);  
            }  
        } catch (InterruptedException e) {  
            System.out.println(Thread.currentThread().getName() + " (" +this.getState()+") catch InterruptedException.");  
        }  
    }  
}
```

```
public class Demo1 {
```

```
    public static void main(String[] args) {  
        try {  
            Thread t1 = new MyThread("t1"); // 新建“线程t1”  
            System.out.println(t1.getName() + " (" +t1.getState()+") is new.");  
  
            t1.start(); // 启动“线程t1”  
            System.out.println(t1.getName() + " (" +t1.getState()+") is started.");  
  
            // 主线程休眠300ms, 然后主线程给t1发“中断”指令。  
            Thread.sleep(300);  
            t1.interrupt();  
            System.out.println(t1.getName() + " (" +t1.getState()+") is interrupted.");  
  
            // 主线程休眠300ms, 然后查看t1的状态。  
            Thread.sleep(300);  
            System.out.println(t1.getName() + " (" +t1.getState()+") is interrupted now.");  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
t1 (NEW) is new.  
t1 (RUNNABLE) is started.  
t1 (RUNNABLE) loop 1  
t1 (RUNNABLE) loop 2  
t1 (TIMED_WAITING) is interrupted.  
t1 (RUNNABLE) catch InterruptedException.  
t1 (TERMINATED) is interrupted now.
```

The deprecated `stop()`, `suspend()`, and `resume()` Methods

NOTE: The `Thread` class also contains the `stop()`, `suspend()`, and `resume()` methods. As of Java 2, these methods are *deprecated* (or *outdated*) **because they are known to be inherently unsafe**. You should assign `null` to a `Thread` variable to indicate that it is stopped rather than use the `stop()` method.

`stop()`方法太过于暴力，会强行把执行一半的线程终止。这样就不会保证线程的资源正确释放，通常是没有给与线程完成资源释放工作的机会，因此会导致程序工作在不确定的状态下。

导致死锁！

比较通用的终止线程的形式

```
@Override
public void run() {
    try {
        // 1. isInterrupted() 保证，只要中断标记为true就终止线程。
        while (!isInterrupted()) {
            // 执行任务...
        }
    } catch (InterruptedException ie) {
        // 2. InterruptedException异常保证，当InterruptedException异常产生时，线程被终止。
    }
}
```

而interrupt()常常被用来终止“阻塞状态”线程

• interrupt能否中断处于非阻塞状态的线程?

```
public class ThreadTest {
```

```
    /**
     * @param args
     */
    public static void main(String[] args) {
        WorkThread wThread = new WorkThread();
        wThread.start();
        System.out.println("开始调用wThread.interrupt()");
        wThread.interrupt();
        System.out.println("结束调用wThread.interrupt()");
    }
}
```

```
class WorkThread extends Thread {
    public void run() {
        for (int i = 0; i < Byte.MAX_VALUE;) {
            System.out.println("工作线程运行" + (++i));
        }
    }
}
```

运行结果:

开始调用wThread.interrupt()
工作线程运行1
结束调用wThread.interrupt()
工作线程运行2
工作线程运行3
工作线程运行4
工作线程运行5
工作线程运行6
工作线程运行7
工作线程运行8
工作线程运行9
工作线程运行10
工作线程运行11
工作线程运行12

结论: interrupt无法中断非阻塞状态的线程。

但是, 可以换个思路, 既然之前提过interrupt方法会设置线程的中断状态, 那么可以通过isInterrupt()来进行判断, 从而中断线程。

• 利用isInterrupt & interrupt来结束线程

```
public class ThreadTest {
```

```
    /**
```

```
     * @param args
```

```
     */
```

```
    public static void main(String[] args) {
```

```
        WorkThread wThread = new WorkThread();
```

```
        wThread.start();
```

```
        System.out.println("开始调用wThread.interrupt()");
```

```
        wThread.interrupt();
```

```
        System.out.println("结束调用wThread.interrupt()");
```

```
    }
```

```
}
```

```
class WorkThread extends Thread {
```

```
    public void run() {
```

```
        for (int i = 0; i < Byte.MAX_VALUE && isInterrupted(); ) {
```

```
            System.out.println("工作线程运行" + (++i));
```

```
        }
```

```
    }
```

```
}
```

If a thread is currently in the Ready or Running state, its interrupted flag is set 并没有抛出异常

运行结果:

开始调用wThread.interrupt()

结束调用wThread.interrupt()

interrupt配合isInterrupted() 能够中断非阻塞状态的线程。

• interrupt能否中断阻塞状态的线程？

```
public class ThreadTest {  
  
    /**  
     * @param args  
     */  
    public static void main(String[] args) {  
        WorkThread wThread = new WorkThread();  
        wThread.start();  
        System.out.println("开始调用wThread.interrupt()");  
        wThread.interrupt();  
        System.out.println("结束调用wThread.interrupt()");  
    }  
}
```

运行结果：

开始调用wThread.interrupt()
工作线程sleep
结束调用wThread.interrupt()
工作线程InterruptedException
工作线程运行完毕

```
class WorkThread extends Thread {  
    public void run() {  
        System.out.println("工作线程sleep");  
        for (int i = 0; i < Byte.MAX_VALUE; ) {  
            try {  
                sleep(10 * 1000);  
                System.out.println("工作线程运行"++i);  
            } catch (InterruptedException e) {  
                System.out.println("工作线程InterruptedException")  
                break;  
            }  
        }  
        System.out.println("工作线程运行完毕");  
    }  
}
```

从程序到运行结果来看，当工作线程进入sleep（即阻塞）的时候，调用interrupt方法，将会促使线程抛出异常。
结论：interrupt能够中断阻塞状态的线程。

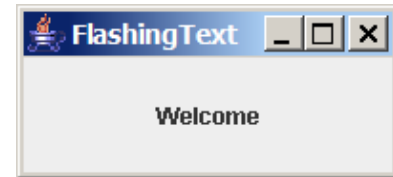
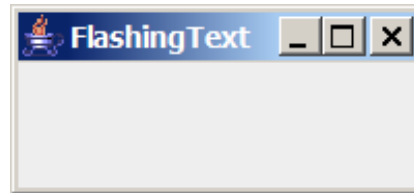
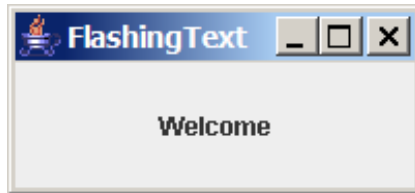
是否daemon线程

- Thread有个是否为daemon线程的属性，相关方法为：
 - `public final void setDaemon(Boolean on)`
 - `public final boolean isDaemon()`
- 一般程序只有在所有线程都结束时才退出，但daemon线程除外，当整个程序中剩下的都是daemon线程时，程序就会退出。
- Daemon一般为辅助线程，当主线程退出时，它就没有存在的意义了，如垃圾回收线程。

Thread Priority

- Each thread is assigned a default priority of `Thread.NORM_PRIORITY`. You can reset the **priority** using `setPriority(int priority)`.
- Some constants for priorities include
`Thread.MIN_PRIORITY`
`Thread.MAX_PRIORITY`
`Thread.NORM_PRIORITY`

Example: Flashing Text



FlashingText

Run

GUI Event Dispatcher Thread

GUI event handling and painting code executes in a single thread, called the *event dispatcher thread*.

This **ensures** that each event handler finishes executing before the next one executes and the painting isn't interrupted by events.

因为大多数SwingAPI是非线程安全的，也就是说不能在任意地方调用，它应该只在EDT中调用。Swing的线程安全靠事件队列和EDT来保障。

Swing将GUI请求放入一个事件队列中执行。

通过EDT，使得不具备线程安全的Swing函数库避开了并发访问的问题。

invokeLater and invokeAndWait

In certain situations, you need to run the code in the event dispatcher thread **to avoid possible deadlock**. You can use the static methods, invokeLater and invokeAndWait, in the javax.swing.SwingUtilities class to **run the code in the event dispatcher thread**.

You must put this code in the run method of a Runnable object and specify the Runnable object as the argument to invokeLater and invokeAndWait. The invokeLater method returns immediately, without waiting for the event dispatcher thread to execute the code. The invokeAndWait method is just like invokeLater, except that invokeAndWait doesn't return until the event-dispatching thread has executed the specified code.

Launch Application from Main Method

So far, you have launched your GUI application from the main method by creating a frame and making it visible. This works fine for most applications. In certain situations, however, it could cause problems. To avoid possible thread deadlock, you should launch GUI creation from the event dispatcher thread as follows:

```
public static void main(String[] args) {  
    SwingUtilities.invokeLater(new Runnable() {  
        public void run() {  
            // Place the code for creating a frame and setting it properties  
        }  
    });  
}
```

EventDispatcherThreadDemo

```

3 public class EventDispatcherThreadDemo extends JApplet {
4     public EventDispatcherThreadDemo() {
5         add(new JLabel("Hi, it runs from an event dispatcher thread"));
6     }
7
8     /** Main method */
9     public static void main(String[] args) {
0         SwingUtilities.invokeLater(new Runnable() {
1             public void run() {
2                 JFrame frame = new JFrame("EventDispatcherThreadDemo");
3                 frame.add(new EventDispatcherThreadDemo());
4                 frame.setLocationRelativeTo(null); // Center the frame
5                 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
6                 frame.setSize(200, 200);
7                 frame.setVisible(true);
8             }
9         });
0     }
1 }
2

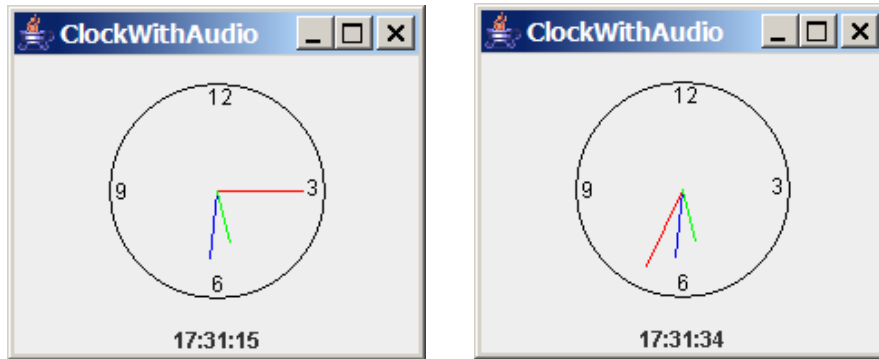
```

Swing编程时应该注意：

1. 从其他线程访问UI组件及其事件处理器会导致界面更新和绘制错误。——**必须通过EDT刷新组件**
2. 在EDT上执行耗时任务会使程序失去响应，这会使GUI事件阻塞在队列中得不到处理。——**不能在EDT中执行其他耗时操作**
3. 应使用独立的任务线程来执行耗时计算或输入输出密集型任务，比如同数据库通信、访问网站资源、读写大树据量的文件。——**耗时操作应放到独立的任务线程中，通过SwingWorker启动**

总之，**任何干扰或延迟UI事件的处理只应该出现在独立任务线程中；**

Case Study: Clock with Audio (Optional)



The example creates an applet that displays a running clock and announces the time at one-minute intervals. For example, if the current time is 6:30:00, the applet announces, "six o'clock thirty minutes a.m." If the current time is 20:20:00, the applet announces, "eight o'clock twenty minutes p.m." Also add a label to display the digital time.

[ClockWithAudio](#)

Run

```

private class TimerListener implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent e) {
        clock.setCurrentTime();
        clock.repaint();
        jlblDigitTime.setText(clock.getHour() + ":" + clock.getMinute()
            + ":" + clock.getSecond());
        if (clock.getSecond() == 0)
            announceTime(clock.getHour(), clock.getMinute());
    }
}

```

```

/** Announce the current time at every minute */
public void announceTime(int hour, int minute) {
    // Announce hour
    hourAudio[hour % 12].play();

    try {
        // Time delay to allow hourAudio play to finish
        Thread.sleep(1500);

        // Announce minute
        minuteAudio[minute].play();

        // Time delay to allow minuteAudio play to finish
        Thread.sleep(1500);
    }
    catch (InterruptedException ex) {
    }

    // Announce am or pm
    if (hour < 12)

```

.....

耗时的
任务在
同一个
EDT中

Run Audio on Separate Thread

When you run the preceding program, you will notice that the second hand does not display at the first, second, and third seconds of the minute. This is because sleep(1500) is invoked twice in the announceTime() method, which takes three seconds to announce the time at the beginning of each minute. Thus, **the next action event is delayed** for three seconds during the first three seconds of each minute. As a result of this delay, the time is not updated and the clock was **not repainted** for these three seconds. **To fix this problem, you should announce the time on a separate thread.** This can be accomplished by modifying the announceTime method.

[ClockWithAudioOnSeparateThread](#)

Run

```

53  }
54  public void actionPerformed(ActionEvent e) {
55      clock.setCurrentTime();
56      clock.repaint();
57      jlblDigitTime.setText(clock.getHour() + ":" + clock.getMinute()
58          + ":" + clock.getSecond());
59      if (clock.getSecond() == 0)
60          announceTime(clock.getHour(), clock.getMinute());
61  }
62  }
63
64  /** Announce the current time at every minute */
65  public void announceTime(int h, int m) {
66      new Thread(new AnnounceTimeOnSeparateThread(h, m)).start();
67  }
68
69  /** Inner class for announcing time */
70  class AnnounceTimeOnSeparateThread implements Runnable {
71      private int hour, minute;
72
73      /** Get Audio clips */
74      public AnnounceTimeOnSeparateThread(int hour, int minute) {
75          this.hour = hour;
76          this.minute = minute;
77      }
78
79      public void run() {
80          // Announce hour
81          hourAudio[hour % 12].play();
82
83          try {
84              // Time delay to allow hourAudio play to finish
85              Thread.sleep(1500);
86

```

耗时的任务
用单独的线程

共享内存及可能存在的问题

- 每个线程表示一条单独的执行流，有自己的程序计数器，有自己的栈，但线程之间可以共享内存，可以访问和操作相同的对象。

```
private static int shared = 0;
private static void incrShared() {
    shared++;
}
static class ChildThread extends Thread{
    List<String> list;
    public ChildThread(List<String>list) {
        this.list = list;
    }
    @Override
    public void run() {
        incrShared();
        list.add(Thread.currentThread().getName());
    }
}
```

共享内存及可能存在的问题

```
public static void main(String[] args) throws InterruptedException{  
    List<String> list = new ArrayList<String>();  
    Thread t1 = new ChildThread(list);  
    Thread t2 = new ChildThread(list);  
    t1.start();  
    t2.start();  
    t1.join();  
    t2.join();  
    System.out.println(shared);  
    System.out.println(list);  
}
```

```
<terminated> ShareMemoryDe  
2  
[Thread-0, Thread-1]
```

- 例子中有三个执行流：main和两个ChildThread线程
- 不同的执行流可以访问和操作相同的变量
- 不同的执行流可以执行相同的代码(incrShared)
- 多条执行流执行相同的代码时，每条执行流都有单独的栈，方法中的局部变量或参数都有自己的一份。

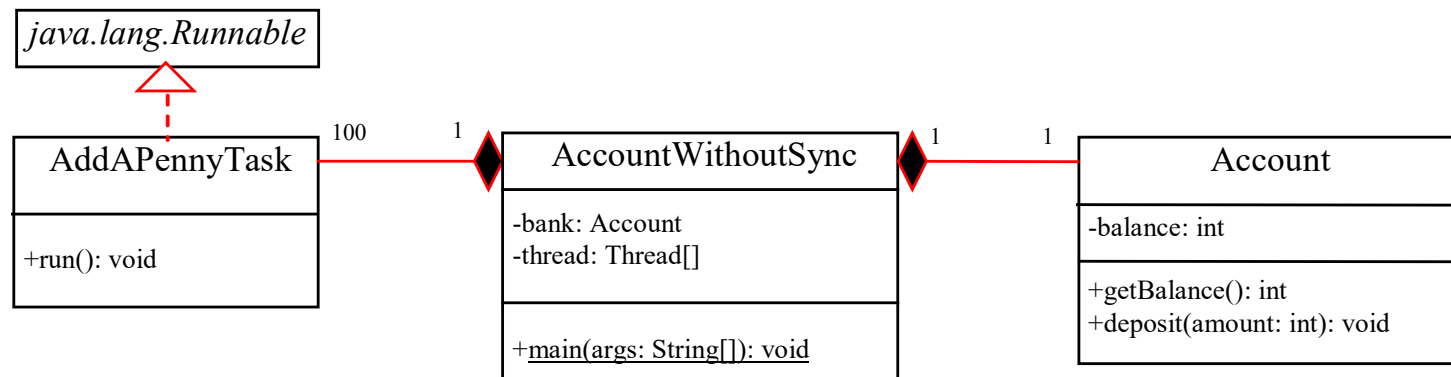
Thread Synchronization

A shared resource may be corrupted if it is accessed simultaneously by multiple threads. For example, two unsynchronized threads accessing the same bank account may cause conflict.

Step	balance	thread[i]	thread[j]
1	0	<code>newBalance = bank.getBalance() + 1;</code>	
2	0		<code>newBalance = bank.getBalance() + 1;</code>
3	1	<code>bank.setBalance(newBalance);</code>	
4	1		<code>bank.setBalance(newBalance);</code>

Example: Showing Resource Conflict

- Objective: Write a program that demonstrates the problem of resource conflict. Suppose that you create and launch one hundred threads, each of which adds a penny to an account. Assume that the account is initially empty.



```
Command Prompt
C:\book>java AccountWithoutSync
What is balance ? 5

C:\book>java AccountWithoutSync
What is balance ? 4

C:\book>java AccountWithoutSync
What is balance ? 7

C:\book>
```

[AccountWithoutSync](#)

Run

Race Condition

What, then, caused the error in the example? Here is a possible scenario:

Step	balance	Task 1	Task 2
1	0	newBalance = balance + 1;	
2	0		newBalance = balance + 1;
3	1	balance = newBalance;	
4	1		balance = newBalance;

The effect of this scenario is that Task 1 did nothing, because in Step 4 Task 2 overrides Task 1's result. Obviously, the problem is that Task 1 and Task 2 are accessing a common resource in a way that causes conflict. This is a common problem known as a *race condition* in multithreaded programs.

A class is said to be *thread-safe* if an object of the class does not cause a race condition in the presence of multiple threads. As demonstrated in the preceding example, the Account class is not thread-safe.

Race Condition

- 如何解决？
 - synchronized 关键词
 - 使用显式锁
 - 使用原子变量

内存可见性问题

```
3 public class VisibilityDemo {
4     private static boolean shutdown=false;
5     static class HelloThread extends Thread{
6         @Override
7         public void run(){
8             while(!shutdown){
9                 //do nothing
10            }
11        }
12    }
13    public static void main(String[] args) throws InterruptedException{
14        new HelloThread().start();
15        Thread.sleep(1000);
16        shutdown = true;
17        System.out.println("exit main");
18    }
19 }
```

为什么永远
不结束？

Problems Javadoc Declaration Console
VisibilityDemo [Java Application] C:\Program Files\Java\jdk1.8.0_141\jre\bin\javaw.exe (2020年12月28日 下午11:02:12)
exit main

如何解决？使用volatile关键词；使用synchronized或显式锁

The `synchronized` keyword

To avoid race conditions, more than one thread must be prevented from simultaneously entering certain part of the program, known as **critical region**.

The critical region in the Listing 29.7 is the entire deposit method. You can use the `synchronized` keyword to synchronize the method so **that only one thread can access the method at a time**. There are several ways to correct the problem in Listing 29.7, one approach is to make Account thread-safe by adding the `synchronized` keyword in the deposit method in Line 45 as follows:

```
public synchronized void deposit(double amount)
```

Synchronizing Instance Methods and Static Methods

A synchronized method acquires a lock before it executes.

In the case of **an instance method**, **the lock is on the object** for which the method was invoked.

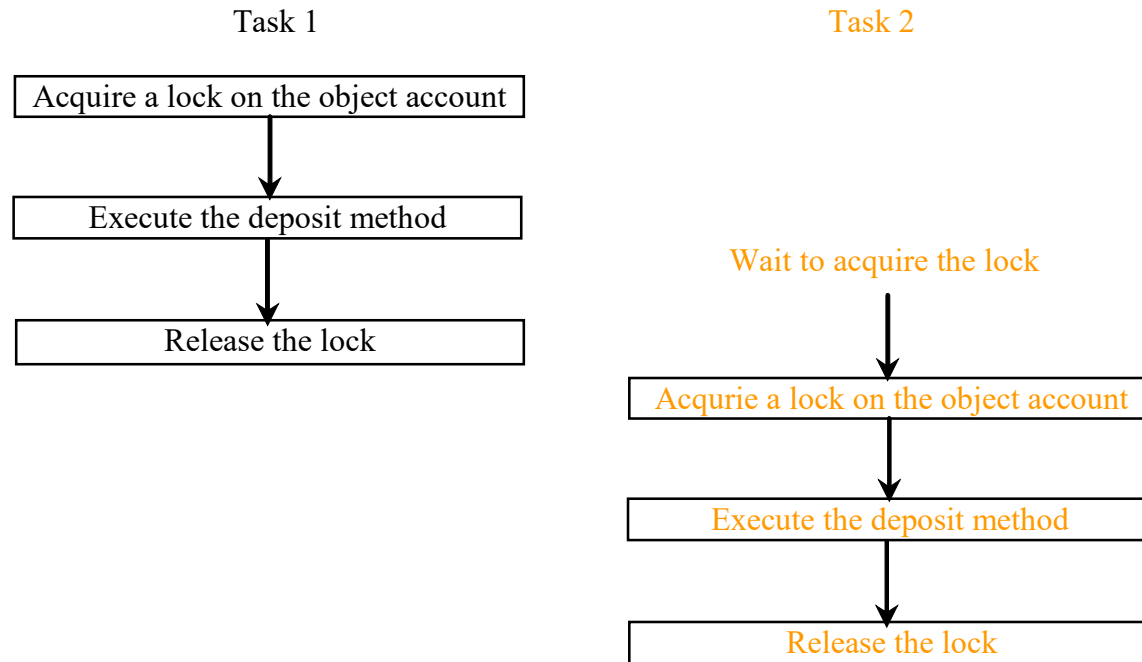
In the case of **a static method**, **the lock is on the class**.

If one thread invokes a synchronized instance method (respectively, static method) on an object, **the lock of that object (respectively, class) is acquired first**, then the method is executed, and finally the lock is released.

Another thread invoking the same method of that object (respectively, class) is **blocked** until the lock is released.

Synchronizing Instance Methods and Static Methods

With the deposit method synchronized, the preceding scenario cannot happen. If Task 2 starts to enter the method, and Task 1 is already in the method, Task 2 is blocked until Task 1 finishes the method.



Synchronizing Statements

Invoking a synchronized instance method of an object acquires a lock on the object, and invoking a synchronized static method of a class acquires a lock on the class. **A synchronized statement can be used to acquire a lock on any object, not just *this* object, when executing a block of the code in a method.** This block is referred to as a *synchronized block*. The general form of a synchronized statement is as follows:

```
synchronized (expr) {  
    statements;  
}
```

The expression *expr* must evaluate to an object reference. If the object is already locked by another thread, the thread is blocked until the lock is released. When a lock is obtained on the object, the statements in the synchronized block are executed, and then the lock is released.

Synchronizing Statements vs. Methods

Any synchronized instance method can be converted into a synchronized statement. Suppose that the following is a synchronized instance method:

```
public synchronized void xMethod() {  
    // method body  
}
```

This method is equivalent to

```
public void xMethod() {  
    synchronized (this) {  
        // method body  
    }  
}
```

```
public class StaticMethod{  
    ...  
    public static synchronized void xMethod() {  
        // method body  
    }  
}
```

This method is equivalent to

```
public class StaticMethod{  
    ...  
    public static void xMethod() {  
        synchronized (StaticMethod.class) {  
            // method body  
        }  
    }  
}
```

synchronized同步的对象可以是任意对象

```
public class Counter {  
    private int count;  
    private Object lock = new Object();  
    public void incr() {  
        synchronized(lock) {  
            count++;  
        }  
    }  
    public int getCount() {  
        synchronized(lock) {  
            return count;  
        }  
    }  
}
```

More for synchronized

- 可重入性
 - Synchronized一个重要的特征：可重入性。即：对同一个线程，它在获得锁之后，在调用其他需要同样锁的代码时，可以直接调用。
 - 如：在一个synchronized实例方法内，可以直接调用其他synchronized实例方法
 - 并不是所有的锁都是可重入的
- 可重入是通过记录锁的持有线程和持有数量来实现的。
 - 调用synchronized保护的代码时，先检查对象是否被锁，如果是，检查是否是当前线程锁定，如果是，则增加持有数量；如果不是，则加入等待队列。释放锁时，减少持有数量，当数量=0时，释放锁。

More for synchronized

- 内存可见性
 - 简单变量操作需要synchronized吗？

```
public class Switcher {  
    private boolean on;  
    public boolean isOn() {  
        return on;  
    }  
    public void setOn(boolean on) {  
        this.on = on;  
    }  
}
```

- 多线程情况下，可能有内存可见性问题
- Synchronized可保证内存可见性，释放锁时，所有的写入都会写回内存，获得锁后，会从内存中读最新数据。

More for synchronized

- 内存可见性
 - 更轻量级的方法: `volatile`
 - 保证读写到内存最新值

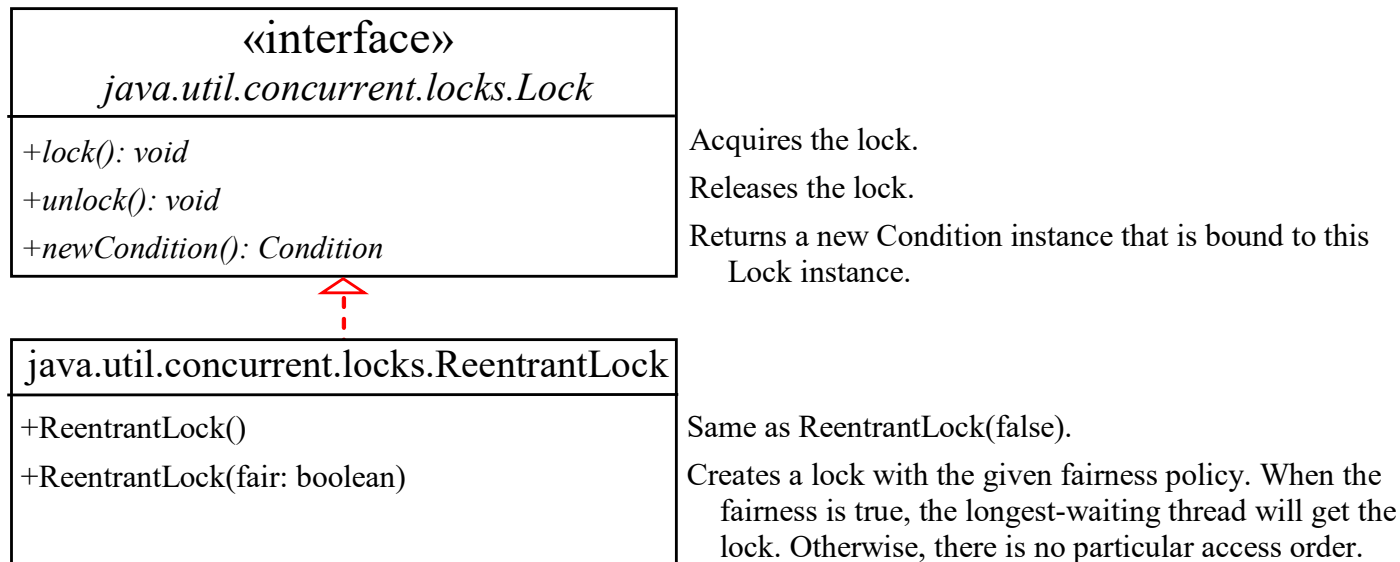
```
public class Switcher {  
    private volatile boolean on;  
    public boolean isOn() {  
        return on;  
    }  
    public void setOn(boolean on) {  
        this.on = on;  
    }  
}
```

Synchronization Using Locks

A synchronized instance method implicitly acquires a lock on the instance before it executes the method.

JDK 1.5 enables you to use locks explicitly. **The new locking features are flexible and give you more control for coordinating threads.**

1. A lock is an instance of the **Lock** interface, which declares the methods for acquiring and releasing locks, as shown in Figure 29.14.
2. A lock may also use the [newCondition\(\)](#) method to create any number of [Condition](#) objects, which can be used for thread communications.



Fairness Policy

ReentrantLock is a concrete implementation of Lock for creating **mutual exclusive locks**.

```
java.util.concurrent.locks.ReentrantLock  
+ReentrantLock()  
+ReentrantLock(fair: boolean)
```

You can create a lock with the specified fairness policy.
True fairness policies guarantee the longest-wait thread to obtain the lock first.

False fairness policies grant a lock to a waiting thread without any access order. (default)

Programs using fair locks accessed by many threads may have **poor overall performance** than those using the default setting, but have smaller variances in times to obtain locks and guarantee lack of starvation.

Lock的公平锁和非公平锁

```
Lock lock=new ReentrantLock(true);//公平锁  
Lock lock=new ReentrantLock(false);//非公平锁
```

- 公平锁指的是线程获取锁的顺序是按照加锁顺序来的，而非公平锁指的是抢锁机制，先lock的线程不一定先获得锁。

Example: Using Locks

This example revises AccountWithoutSync.java in Listing 29.7 to synchronize the account modification using explicit locks.

```
// An inner class for account
public static class Account {
    private static Lock lock = new ReentrantLock(); // Create a lock
    private int balance = 0;

    public int getBalance() {
        return balance;
    }

    public void deposit(int amount) {
        lock.lock(); // Acquire the lock

        try {
            int newBalance = balance + amount;

            // This delay is deliberately added to magnify the
            // data-corruption problem and make it easy to see.
            Thread.sleep(5);

            balance = newBalance;
        }
        catch (InterruptedException ex) {
        }
        finally {
            lock.unlock(); // Release the lock
        }
    }
}
```

AccountWithSyncUsingLock

Run

```
public void read() {
    try {
        try {
            lock.readLock().lock();
            System.out.println("获得读锁" + Thread.currentThread().getName()
                               + " " + System.currentTimeMillis());
            Thread.sleep(10000);
        } finally {
            lock.readLock().unlock();
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

public void write() {
    try {
        try {
            lock.writeLock().lock();
            System.out.println("获得写锁" + Thread.currentThread().getName()
                               + " " + System.currentTimeMillis());
            Thread.sleep(10000);
        } finally {
            lock.writeLock().unlock();
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

Lock类有读锁和写锁，读读共享，写写互斥，读写互斥

线程间的协作

- 生成者/消费者协作模式
- 同时开始
 - 要求多个线程同时开始
- 等待结束
 - 主线程将任务分解为若干个子任务，为每个子任务创建一个线程，主线程在继续执行其他任务之前需要等待每个子任务执行完毕
- 异步结果
 - 将子任务的管理封装为异步调用，异步调用马上返回，但返回的不是最终结果，而是一个一般称为Future的对象，可通过它随后获得最终的结果
- 集合点
 - 等所有线程到一个集合点后，交换数据和计算结果，在下一次迭代等

Cooperation Among Threads

The conditions can be used to facilitate communications among threads.

A thread can specify what to do under a certain condition.

Conditions are objects created by invoking the `newCondition()` method on a `Lock` object. Once a condition is created, **you can use its `await()`, `signal()`, and `signalAll()` methods for thread communications**, as shown in Figure 29.15. **[都需要获得锁后执行]**

The `await()` method causes the current thread to **wait** until the condition is signaled.

会释放锁

The `signal()` method wakes up one waiting thread, and the `signalAll()` method wakes all waiting threads.

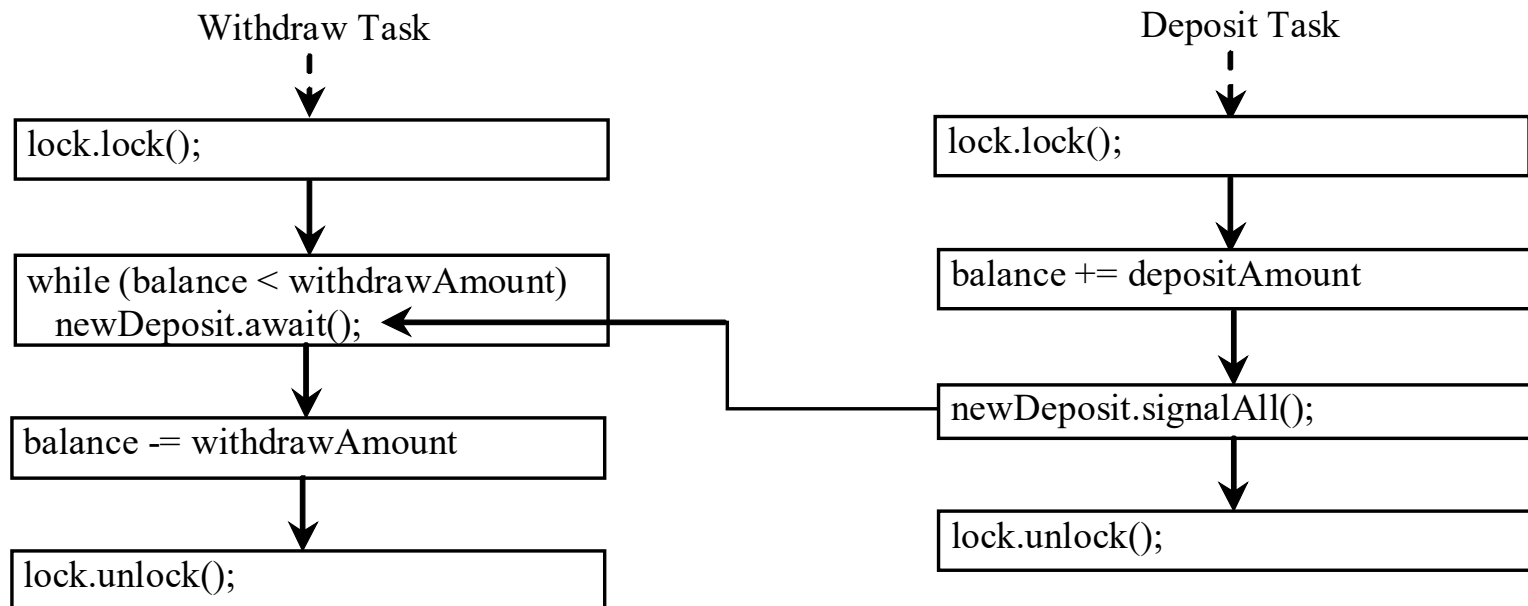
«interface» <i>java.util.concurrent.Condition</i>	
+ <i>await(): void</i>	Causes the current thread to wait until the condition is signaled.
+ <i>signal(): void</i>	Wakes up one waiting thread.
+ <i>signalAll(): Condition</i>	Wakes up all waiting threads.

wait的具体过程

- 把当前线程放入条件等待队列，释放对象锁，阻塞等待，线程状态变为WAITING或TIMED_WAITING。
- 等待时间到或被其他线程调用notify/notifyAll从条件队列中移除，这是，要重新竞争对象锁
 - 如果能获得锁，线程变为RUNNABLE，并从wait 调用中返回
 - 否则，线程加入对象锁等待队列，线程变为BLOCKED，只有在获得锁后才会从wait调用中返回。

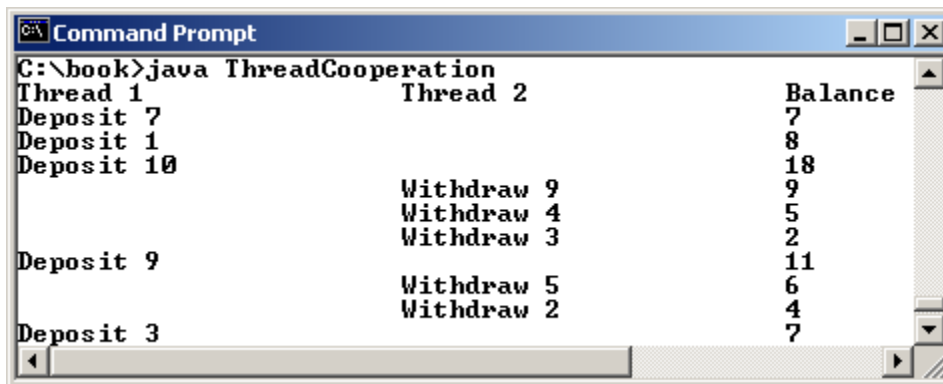
Cooperation Among Threads

To synchronize the operations, use a lock with a condition: newDeposit (i.e., new deposit added to the account). If the balance is less than the amount to be withdrawn, the withdraw task will **wait** for the newDeposit condition. When the deposit task adds money to the account, the task **signals** the waiting withdraw task to try again. The interaction between the two tasks is shown in Figure 29.16.



Example: Thread Cooperation

Write a program that demonstrates thread cooperation. Suppose that you create and launch two threads, one deposits to an account, and the other withdraws from the same account. The second thread has to wait if the amount to be withdrawn is more than the current balance in the account. Whenever new fund is deposited to the account, the first thread notifies the second thread to resume. If the amount is still not enough for a withdrawal, the second thread has to continue to wait for more fund in the account. Assume the initial balance is 0 and the amount to deposit and to withdraw is randomly generated.



```
Command Prompt
C:\book>java ThreadCooperation
Thread 1      Thread 2      Balance
Deposit 7           7
Deposit 1          8
Deposit 10         18
                Withdraw 9    9
                Withdraw 4    5
                Withdraw 3    2
Deposit 9          11
                Withdraw 5    6
                Withdraw 2    4
Deposit 3          7
```

[ThreadCooperation](#)

Run

• 若

```
while (balance < amount) {  
    System.out.println("\t\t\tWait for a deposit");  
    newDeposit.await();  
}
```

用下面的语句代替，会发生什么？

```
if (balance < amount) {  
    System.out.println("\t\t\tWait for a deposit");  
    newDeposit.await();  
}
```

当deposit时，
虽然调用了
signal，但仍然
需要验证
balance是否
>amount

- 一旦线程调用await，则进入等待状态（交出锁），将等待信号的恢复。若忘了调用signal或signalAll，则永远等待。
- 条件有Lock对象创建。为调用await, signal, signalAll等，必须首先要拥有锁，否则抛出IllegalMonitorStateException异常。

Java's Built-in Monitors (Optional)

Locks and conditions are new in Java 5. **Prior to Java 5**, thread communications are programmed using object's built-in monitors. **Locks and conditions are more powerful and flexible than the built-in monitor.** **For this reason, this section can be completely ignored.** However, if you work with legacy Java code, you may encounter the Java's built-in monitor.

A ***monitor*** is an object with mutual exclusion and synchronization capabilities. **Only one thread can execute a method at a time in the monitor.** A thread enters the monitor by acquiring a lock on the monitor and exits by releasing the lock. ***Any object can be a monitor.*** **An object becomes a monitor once a thread locks it.** **Locking is implemented using the synchronized keyword on a method or a block.** A thread must acquire a lock before executing a synchronized method or block. A thread can wait in a monitor if the condition is not right for it to continue executing in the monitor.

wait(), notify(), and notifyAll()

Use the wait(), notify(), and notifyAll() methods to facilitate communication among threads.

The wait(), notify(), and notifyAll() methods must be called **in a synchronized method or a synchronized block** on the calling **object** of these methods. Otherwise, an IllegalMonitorStateException would occur.

The wait() method lets the thread wait until some condition occurs. When it occurs, you can use the notify() or notifyAll() methods to notify the waiting threads to resume normal execution. The notifyAll() method wakes up all waiting threads, while notify() picks up only one thread from a waiting queue.

Example: Using Monitor

Task 1

```
synchronized (anObject) {  
    try {  
        // Wait for the condition to become true  
        while (!condition)  
            anObject.wait();  
        // Do something when condition is true  
    }  
    catch (InterruptedException ex) {  
        ex.printStackTrace();  
    }  
}
```

resume

Task 2

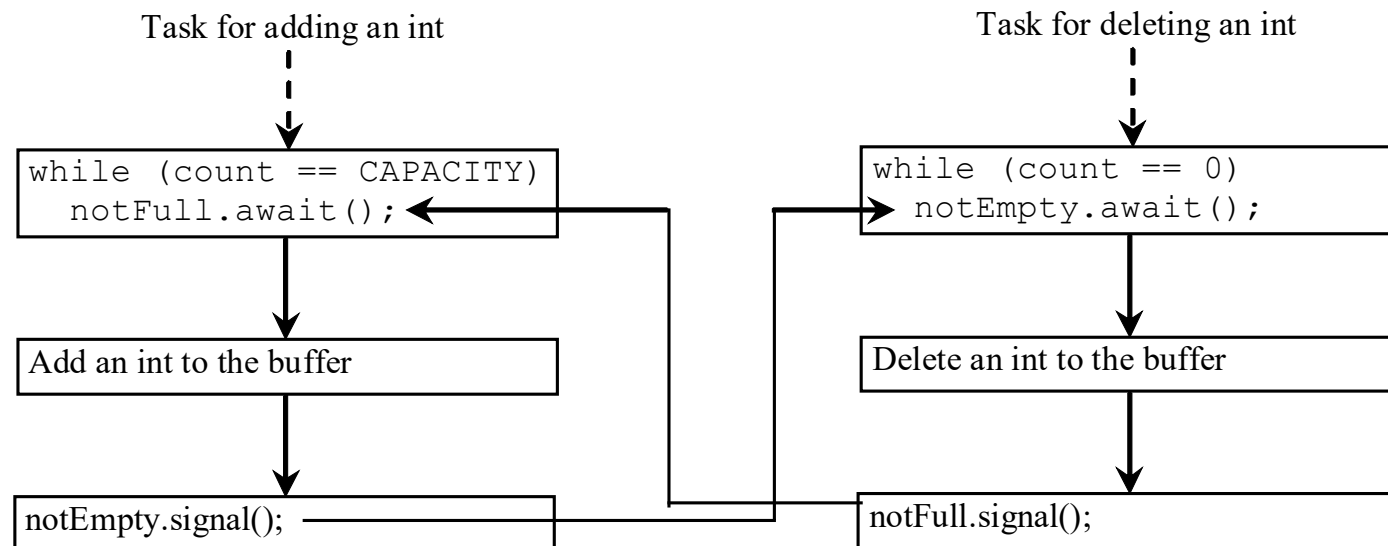
```
synchronized (anObject) {  
    // When condition becomes true  
    anObject.notify(); or anObject.notifyAll();  
    ...  
}
```

- The wait(), notify(), and notifyAll() methods must be called in a **synchronized method or a synchronized block** on the receiving object of these methods. Otherwise, an IllegalMonitorStateException will occur.
- When wait() is invoked, it pauses the thread and simultaneously **releases** the lock on the object. **When the thread is restarted after being notified, the lock is **automatically** reacquired.**
- The wait(), notify(), and notifyAll() methods on an object are analogous to the await(), signal(), and signalAll() methods on a condition.

Case Study: Producer/Consumer (Optional)

Consider the classic Consumer/Producer example. Suppose you use a buffer to store integers. The buffer size is limited. The buffer provides the method write(int) to add an int value to the buffer and the method read() to read and delete an int value from the buffer.

To synchronize the operations, use a lock with two conditions: notEmpty (i.e., buffer is not empty) and notFull (i.e., buffer is not full). When a task adds an int to the buffer, if the buffer is full, the task will wait for the notFull condition. When a task deletes an int from the buffer, if the buffer is empty, the task will wait for the notEmpty condition. The interaction between the two tasks is shown in Figure 29.19.



Case Study: Producer/Consumer (Optional)

Listing 29.10 presents the complete program. The program contains the Buffer class (lines 43-89) and two tasks for repeatedly producing and consuming numbers to and from the buffer (lines 15-41). The write(int) method (line 58) adds an integer to the buffer. The read() method (line 75) deletes and returns an integer from the buffer.

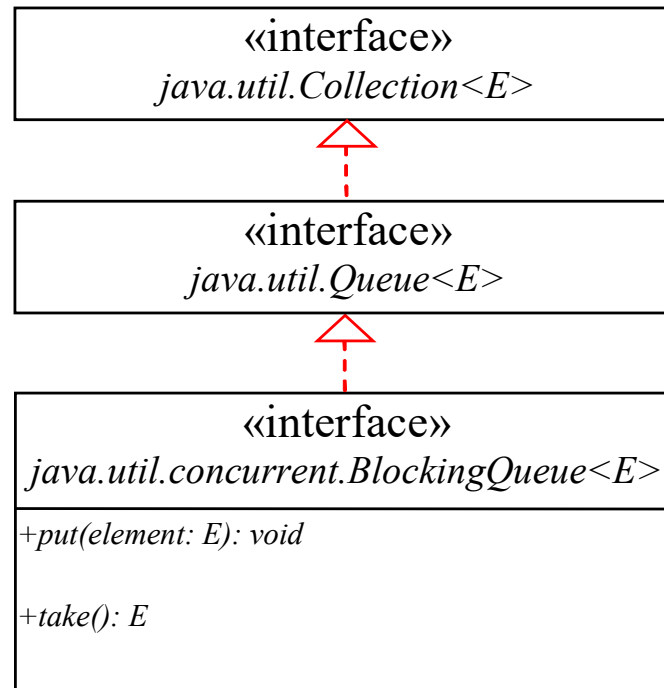
For simplicity, the buffer is implemented using a linked list (lines 48-49). Two conditions notEmpty and notFull on the lock are created in lines 55-56. The conditions are bound to a lock. A lock must be acquired before a condition can be applied. If you use the wait() and notify() methods to rewrite this example, you have to designate two objects as monitors.

[ConsumerProducer](#)

Run

Blocking Queues (Optional)

§ 22.8 introduced queues and priority queues. *A blocking queue* causes a thread to block when you try to add an element to a full queue or to remove an element from an empty queue.

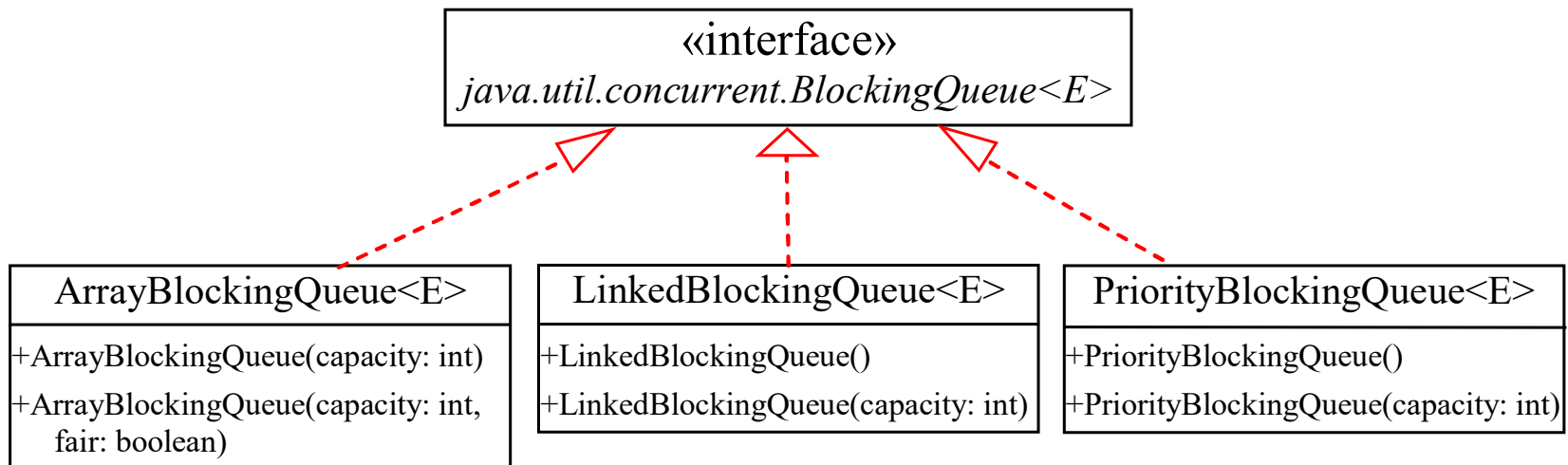


Inserts an element to the tail of the queue.
Waits if the queue is full.

Retrieves and removes the head of this
queue. Waits if the queue is empty.

Concrete Blocking Queues

Three concrete blocking queues [ArrayBlockingQueue](#), [LinkedBlockingQueue](#), and [PriorityBlockingQueue](#) are supported in JDK 1.5, as shown in Figure 29.22. All are in the `java.util.concurrent` package. [ArrayBlockingQueue](#) implements a blocking queue [using an array](#). You have to specify a capacity or an optional fairness to construct an [ArrayBlockingQueue](#). [LinkedBlockingQueue](#) implements a blocking queue [using a linked list](#). You may create an unbounded or bounded [LinkedBlockingQueue](#). [PriorityBlockingQueue](#) is [a priority queue](#). You may create an unbounded or bounded priority queue.



Producer/Consumer Using Blocking Queues

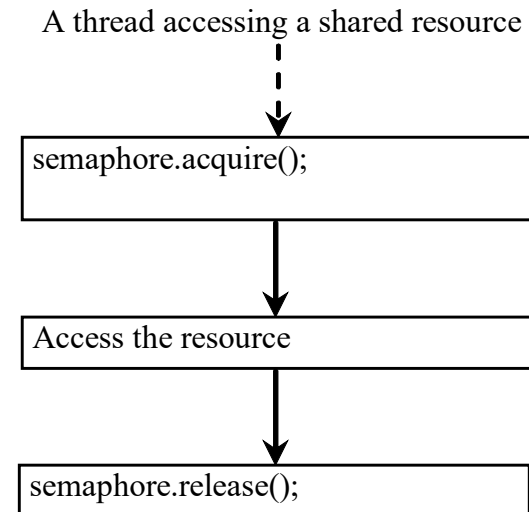
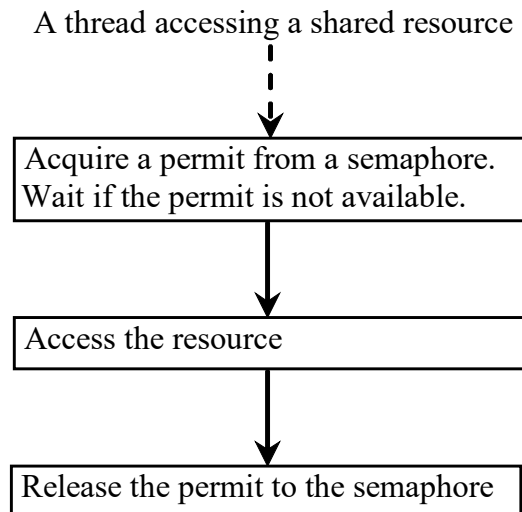
Listing 29.11 gives an example of using an ArrayBlockingQueue to simplify the Consumer/Producer example in Listing 29.11.

```
// A task for adding an int to the buffer
private static class ProducerTask implements Runnable {
    public void run() {
        try {
            int i = 1;
            while (true) {
                System.out.println("Producer writes " + i);
                buffer.put(i++); // Add any value to the buffer, say, 1
                // Put the thread into sleep
                Thread.sleep((int) (Math.random() * 10000));
            }
        } catch (InterruptedException ex) {
            ex.printStackTrace();
        }
    }
}

// A task for reading and deleting an int from the buffer
private static class ConsumerTask implements Runnable {
    public void run() {
        try {
            while (true) {
                System.out.println("\t\t\tConsumer reads " + buffer.take());
                // Put the thread into sleep
                Thread.sleep((int) (Math.random() * 10000));
            }
        } catch (InterruptedException ex) {
            ex.printStackTrace();
        }
    }
}
```

Semaphores (Optional)

Semaphores can be used to restrict the number of threads that access a shared resource. Before accessing the resource, a thread must acquire a permit from the semaphore. After finishing with the resource, the thread must return the permit back to the semaphore, as shown in Figure 29.29.



Creating Semaphores

To create a semaphore, you have **to specify the number of permits** with an optional fairness policy, as shown in Figure 29.29. **A task acquires a permit by invoking the semaphore's acquire() method and releases the permit by invoking the semaphore's release() method.** Once a permit is acquired, the total number of available permits in a semaphore is reduced by 1. Once a permit is released, the total number of available permits in a semaphore is increased by 1.

java.util.concurrent.Semaphore	
+Semaphore(numberOfPermits: int)	Creates a semaphore with the specified number of permits. The fairness policy is false.
+Semaphore(numberOfPermits: int, fair: boolean)	Creates a semaphore with the specified number of permits and the fairness policy.
+acquire(): void	Acquires a permit from this semaphore. If no permit is available, the thread is blocked until one is available.
+release(): void	Releases a permit back to the semaphore.

AccountWithSynUsingSemaphore

```

// An inner class for account
public static class AccountWithSemaphore {
    private static Semaphore semaphore= new Semaphore(1); // Create a lock
    private int balance = 0;

    public int getBalance() {
        return balance;
    }

    public void deposit(int amount) {

        try {
            semaphore.acquire();
            int newBalance = balance + amount;

            // This delay is deliberately added to magnify the
            // data-corruption problem and make it easy to see.
            Thread.sleep(5);

            balance = newBalance;
        }
        catch (InterruptedException ex) {
        }
        finally {
            semaphore.release();
        }
    }
}

```

线程协作：同时开始

- 一个主线程，N个子线程，每个子线程模拟一个运动员，协作的共享变量为开始信号。

```
static class FireFlag{  
    private volatile boolean fired = false;  
    public synchronized void waitForFire() throws InterruptedException  
        while(!fired) {  
            wait();  
        }  
    }  
    public synchronized void fire() {  
        this.fired = true;  
        notifyAll();  
    }  
}
```

```

static class Racer extends Thread{
    FireFlag fireFlag;
    public Racer(FireFlag fireFlag){
        this.fireFlag = fireFlag;
    }
    @Override
    public void run(){
        try{
            this.fireFlag.waitForFire();
            System.out.println("start run "
                               + Thread.currentThread().getName());
        } catch (InterruptedException ex){}
    }
}

```

```

public static void main(String[] args) throws Interruptio
    int num = 10;
    FireFlag fireFlag = new FireFlag();
    Thread[] racers = new Thread[num];
    for(int i=0;i<num;i++){
        racers[i] = new Racer(fireFlag);
        racers[i].start();
    }
    Thread.sleep(1000);
    fireFlag.fire();
}

```

线程协作：等待结束

- 使用join让线程等待另一个线程结束，实际上调用了wait

```
public final synchronized void join(long millis)
    throws InterruptedException {
    long base = System.currentTimeMillis();
    long now = 0;

    if (millis < 0) {
        throw new IllegalArgumentException("timeout v
    }

    if (millis == 0) {
        while (isAlive()) {
            wait(0);
        }
    } else {
        ...
    }
}
```

当线程运行结束后，Java系统会调用notifyAll来通知它

- 主线程和各个子线程协作的共享变量为一个数，表示为完成的线程个数。初始值为子线程个数，主线程等待的值变为0。每个子线程结束后，将该值减一，当减为0时，调用notifyAll。
- 用MyLatch来表示这个协作对象

```
static class MyLatch{  
    private int count;  
    public MyLatch(int count) {  
        this.count = count;  
    }  
    public synchronized void await() throws InterruptedException{  
        while(count>0) {  
            wait();  
        }  
    }  
    public synchronized void countDown() {  
        count--;  
        if(count<=0) {  
            notifyAll();  
        }  
    }  
}
```

```

static class Worker extends Thread{
    MyLatch latch;
    public Worker(MyLatch latch) {
        this.latch = latch;
    }
    @Override
    public void run() {
        try{
            //simulate working on task.
            Thread.sleep((int)Math.random()*1000);
            this.latch.countDown();
        } catch (InterruptedException ex) {}
    }
}

public static void main(String[] args) throws InterruptedException
{
    int workerNum = 10;
    MyLatch latch = new MyLatch(workerNum);
    Worker[] workers = new Worker[workerNum];
    for(int i=0; i<workerNum; i++) {
        workers[i] = new Worker(latch);
        workers[i].start();
    }
    latch.await();
    System.out.println("collect worker results");
}

```

能放到MyLatch latch =
new
MyLatch(workerNum)后
吗？

异步结果

- 异步调用返回一个称为Future的对象，可通过它获得最终的结果。
- 在Java中，表示子任务的接口为

```
public interface Callable<V>{  
    V call() throws Exception;  
}
```

为表示异步调用的结果，我们定义一个接口MyFuture

```
public interface MyFuture <V>{  
    V get() throws Exception;  
}
```

- `get()`返回真正的结果，如果还没有完成，则会阻塞直到完成；如果调用过程发生异常，则`get`方法抛出调用过程中的异常
- 主线程调用子线程，定义一个类`MyExecutor`，其中一个方法
- `public <V> MyFuture<V> execute(final Callable<V> task)`

```

static class ExecuteThread<V> extends Thread{
    private V result = null;
    private Exception exception = null;
    private boolean done = false;
    private Callable<V> task;
    private Object lock;
    public ExecuteThread(Callable<V>task, Object lock){
        this.task = task;
        this.lock = lock;
    }
    @Override
    public void run() {
        try{
            result = task.call();
        } catch (Exception ex) {
            exception = ex;
        } finally {
            synchronized(lock) {
                done = true;
                lock.notifyAll();
            }
        }
    }
    public V getResult() {
        return result;
    }
    public boolean isDone() {
        return done;
    }
    public Exception getException() {
        return exception;
    }
}

```

线程执行结束，
done=true，然后通知
等待该lock的线程

```

static class MyExecutor{
    public <V> MyFuture<V> execute(final Callable<V> task){
        final Object lock = new Object();
        final ExecuteThread<V> thread = new ExecuteThread<>(task, lock);
        thread.start();
        MyFuture<V> future = new MyFuture<V>(){
            @Override
            public V get() throws Exception{
                synchronized (lock){
                    while(!thread.isDone()){
                        try{
                            lock.wait();
                        }catch(InterruptedException ex){

                        }
                    }
                    if(thread.getException() != null){
                        throw thread.getException();
                    }
                    return thread.getResult();
                }
            }
        };
        return future;
    }
}

```

如果线程还没有结束，则wait；如果结束了，则可以执行下去，直到thread.getResult();

集合点

- 各个线程分头行动，各自到达一个集合点，当集齐所有线程后，交换数据，再下一步动作。
- 共享变量是什么？
 - 一个数，表示未到达集合点的线程个数
 - 初始值为子线程个数，每个线程到达后，则减一；如果不为0，则表示还有其他线程没有达到，则等待；如果变为0，则表示自己是最后一个达到，则notifyAll唤醒所有的线程。

```

static class AssemblePoint{
    private int n;
    public AssemblePoint(int n){
        this.n = n;
    }
    public synchronized void await() throws InterruptedException{
        if(n>0){
            n--;
            if(n==0){
                notifyAll();
            }else{
                while(n!=0){
                    wait();
                }
            }
        }
    }
}

```

```

static class Tourist extends Thread{
    AssemblePoint ap;
    public Tourist(AssemblePoint ap){
        this.ap = ap;
    }
    @Override
    public void run(){
        try{
            //模拟线程各自运行
            int waitTime = (int) (Math.random()*1000);
            Thread.sleep(waitTime);
            ap.await();
            System.out.println("arrived " +
                Thread.currentThread().getName()+" " + waitTime);
            //集合后的其他操作
        } catch (InterruptedException e){

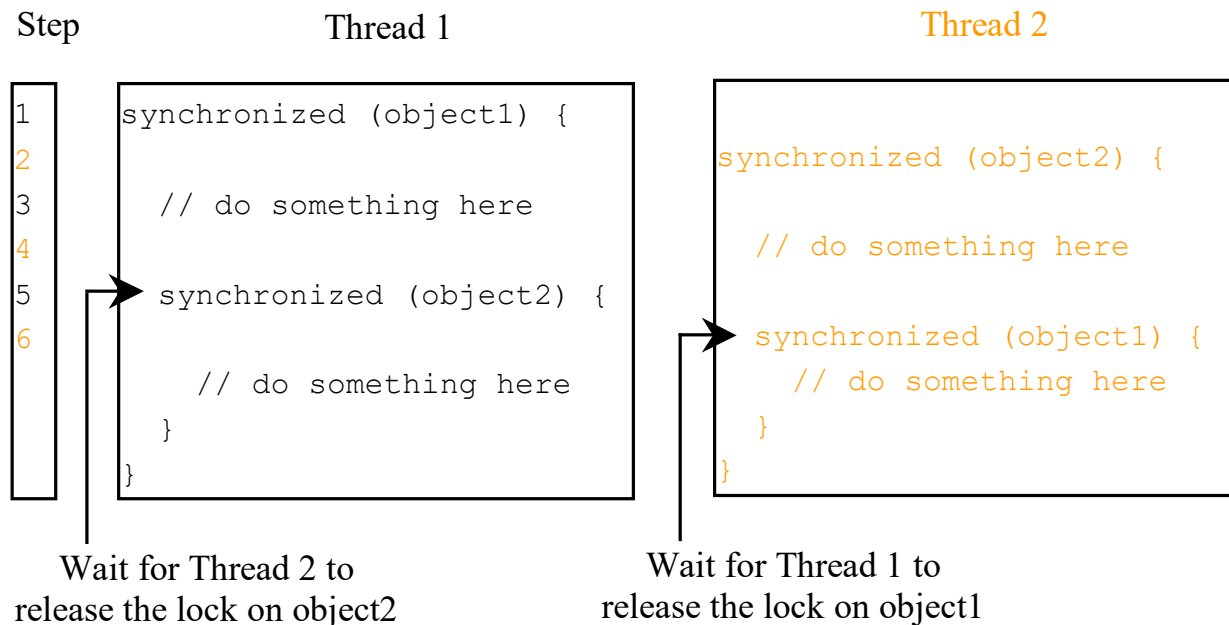
        }
    }
}

public static void main(String[] args) {
    int num = 10;
    Tourist[] threads = new Tourist[num];
    AssemblePoint ap = new AssemblePoint(num);
    for(int i=0;i<num;i++){
        threads[i] = new Tourist(ap);
        threads[i].start();
    }
}

```

Deadlock

Sometimes two or more threads need to acquire the locks on several shared objects. This could cause *deadlock*, in which each thread has the lock on one of the objects and is waiting for the lock on the other object. Consider the scenario with two threads and two objects, as shown in Figure 29.15. Thread 1 acquired a lock on object1 and Thread 2 acquired a lock on object2. Now Thread 1 is waiting for the lock on object2 and Thread 2 for the lock on object1. The two threads wait for each other to release the in order to get the lock, and neither can continue to run.



Preventing Deadlock

Deadlock can be easily avoided by using a simple technique known as **resource ordering**.

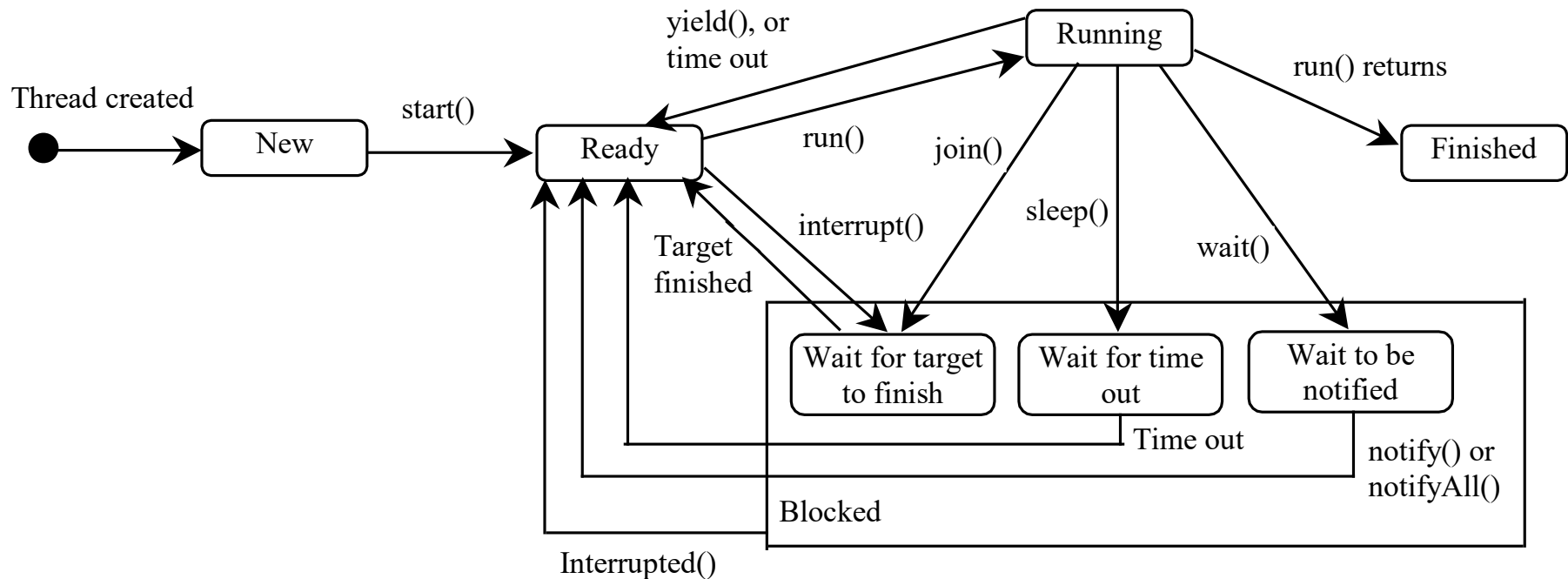
With this technique, you assign an order on all the objects whose locks must be acquired and ensure that each thread acquires the locks in that order. For the example in Figure 29.15, suppose the objects are ordered as object1 and object2. **Using the resource ordering technique, Thread 2 must acquire a lock on object1 first, then on object2.** Once Thread 1 acquired a lock on object1, Thread 2 has to wait for a lock on object1. So Thread 1 will be able to acquire a lock on object2 and no deadlock would occur.

ReentrantLock来解决死锁问题

- tryLock
- tryLock的方法就是试一下，如果能得到锁，就返回真，如果当时得不到，马上就返回假，绝不等。
- tryLock(时间)的用法就是 在规定的时间内设法得到锁。如果在规定的时间内最终不能得到锁，就返回假。

Thread States

A thread can be in one of five states:
New, Ready, Running, Blocked, or
Finished.



Synchronized Collections

The classes in the Java Collections Framework are not thread-safe, i.e., the contents may be corrupted if they are accessed and updated concurrently by multiple threads. You can protect the data in a collection by locking the collection or using synchronized collections.

The Collections class provides six static methods for wrapping a collection into a **synchronized version**, as shown in Figure 29.27. The collections created using these methods are called *synchronization wrappers*.

java.util.Collections	
+ <u>synchronizedCollection(c: Collection): Collection</u>	Returns a synchronized collection.
+ <u>synchronizedList(list: List): List</u>	Returns a synchronized list from the specified list.
+ <u>synchronizedMap(m: Map): Map</u>	Returns a synchronized map from the specified map.
+ <u>synchronizedSet(s: Set): Set</u>	Returns a synchronized set from the specified set.
+ <u>synchronizedSortedMap(s: SortedMap): SortedMap</u>	Returns a synchronized sorted map from the specified sorted map.
+ <u>synchronizedSortedSet(s: SortedSet): SortedSet</u>	Returns a synchronized sorted set.

```
final Collection<E> c;    // Backing Collection
final Object mutex;      // Object on which to synchronize
```

```
SynchronizedCollection(Collection<E> c) {
    this.c = Objects.requireNonNull(c);
    mutex = this;
}
```

```
SynchronizedCollection(Collection<E> c, Object mutex) {
    this.c = Objects.requireNonNull(c);
    this.mutex = Objects.requireNonNull(mutex);
}
```

```
public int size() {
    synchronized (mutex) {return c.size();}
}
public boolean isEmpty() {
    synchronized (mutex) {return c.isEmpty();}
}
public boolean contains(Object o) {
    synchronized (mutex) {return c.contains(o);}
}
public Object[] toArray() {
    synchronized (mutex) {return c.toArray();}
}
public <T> T[] toArray(T[] a) {
    synchronized (mutex) {return c.toArray(a);}
}
```

Vector, Stack, and Hashtable

Invoking synchronizedCollection(Collection c) returns a new Collection object, in which all the methods that access and update the original collection c are synchronized. **These methods are implemented using the synchronized keyword.** For example, the add method is implemented like this:

```
public boolean add(E o) {  
    synchronized (this) { return c.add(o); }  
}
```

The synchronized collections can be safely accessed and modified by multiple threads concurrently.

The methods in java.util.Vector, java.util.Stack, and Hashtable are already synchronized. These are old classes introduced in JDK 1.0. In JDK 1.5, you should use java.util.ArrayList to replace Vector, java.util.LinkedList to replace Stack, and java.util.Map to replace Hashtable. **If synchronization is needed, use a synchronization wrapper.**

Synchronized集合

- 加了synchronized，所有的方法都变为原子操作，客户端调用时，是否绝对安全了？
- 复合操作

```
public class EnhancedMap<K, V> {  
    Map<K, V> map;  
    public EnhancedMap (Map<K, V> map) {  
        this.map = Collections.synchronizedMap(map);  
    }  
    public V putIfAbsent(K key, V value) {  
        V old = map.get(key);  
        if (old != null) {  
            return old;  
        }  
        return map.put(key, value);  
    }  
    public V put(K key, V value) {  
        return map.put(key, value);  
    }  
}
```

可能多个线程会检查，
都发现没有这个键，然
后调用 put，破坏了
putIfAbsent期望保持的
语义

Synchronized集合

- 伪同步

```
public synchronized V putIfAbsent(K key, V value) {  
    V old = map.get(key);  
    if (old != null) {  
        return old;  
    }  
    return map.put(key, value);  
}
```

是否安全？

同步错对象！

```
public V putIfAbsent(K key, V value) {  
    synchronized (map) {  
        V old = map.get(key);  
        if (old != null) {  
            return old;  
        }  
        return map.put(key, value);  
    }  
}
```

Fail-Fast

- The synchronization wrapper classes are thread-safe, but the iterator is *fail-fast*. This means that if you are using an iterator to traverse a collection while the underlying collection is being modified by another thread, then the iterator will immediately fail by throwing `java.util.ConcurrentModificationException`, which is a subclass of `RuntimeException`.

```
public boolean addAll(int index, Collection<? extends E> c) {  
    synchronized (mutex) {return list.addAll(index, c);}  
}  
  
public ListIterator<E> listIterator() {  
    return list.listIterator(); // Must be manually synched by user  
}  
  
public ListIterator<E> listIterator(int index) {  
    return list.listIterator(index); // Must be manually synched by user  
}
```

Fail-Fast

```
private static void startModifyThread(final List<String> list) {
    Thread modifyThread = new Thread(new Runnable() {
        public void run() {
            for(int i=0;i<100;i++){
                list.add("item: " + i);
                try{
                    Thread.sleep((int)Math.random()*10);
                }catch(InterruptedException e){
                }
            }
        }
    });
    modifyThread.start();
}

private static void startIteratorThread(final List<String>list) {
    Thread iteratorThread = new Thread(new Runnable() {
        public void run() {
            while(true) {
                for(String str: list){
                    str
                }
            }
        }
    });
    iteratorThread.start();
}
```

Fail-Fast

```
public static void main(String[] args) {  
    final List<String> list =  
        Collections.synchronizedList(new ArrayList<String>());  
    startIteratorThread(list);  
    startModifyThread(list);  
}
```

Problems	Javadoc	Declaration	Console
<terminated> EnhancedMap [Java Application] C:\Program Files\Java\jdk1.8.0_141\jre\bin\javaw.exe (2020年12月29日 上午12:06:55)			
Exception in thread "Thread-0" java.util.ConcurrentModificationException at java.util.ArrayList\$Itr.checkForComodification(ArrayList.java:901) at java.util.ArrayList\$Itr.next(ArrayList.java:851)			

Fail-Fast

To avoid this error, you need to create a synchronized collection object and acquire a lock on the object when traversing it. For example, suppose you want to traverse a set, you have to write the code like this:

```
Set hashSet = Collections.synchronizedSet(new HashSet());  
synchronized (hashSet) { // Must synchronize it  
    Iterator iterator = hashSet.iterator();  
    while (iterator.hasNext()) {  
        System.out.println(iterator.next());  
    }  
}
```

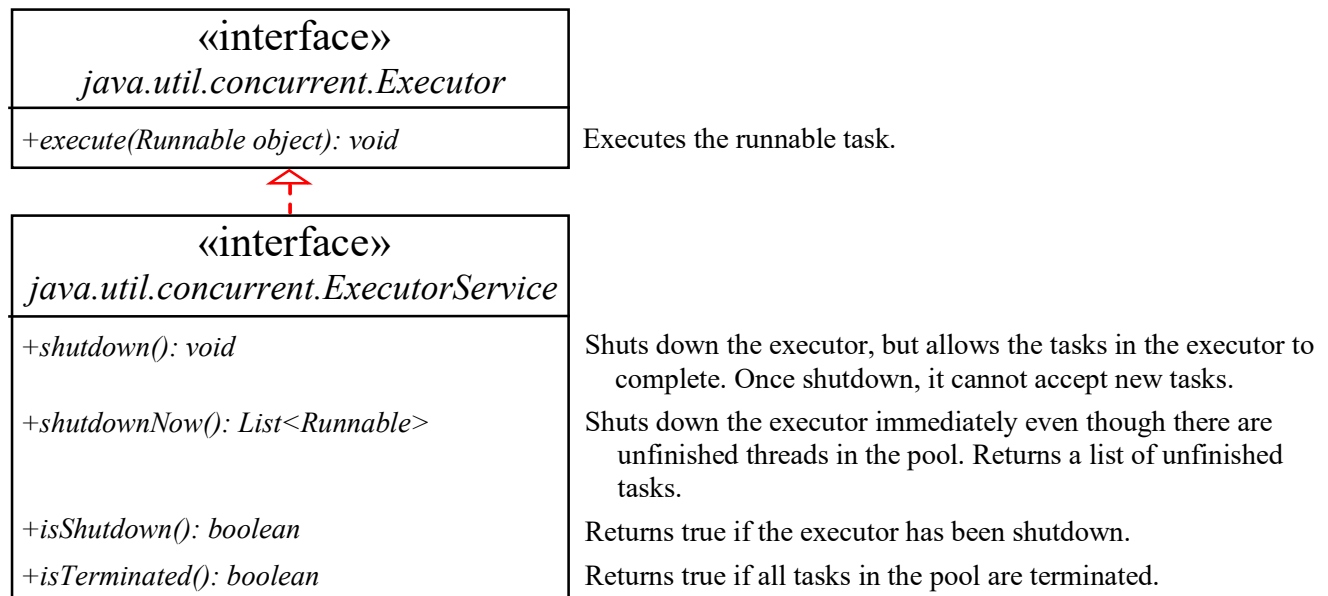
Failure to do so may result in nondeterministic behavior, such as
ConcurrentModificationException.

并发容器

- 同步容器的性能比较低，当并发访问量大的时候，性能较差。Java还有专为并发设计的容器类：
 - CopyOnWriteArrayList
 - ConcurrentHashMap
 - ConcurrentLinkedQueue
 - CocurrentSkpListSet
 -
- 线程安全，且没有使用synochronized，没有迭代问题，直接支持一些复合操作，性能也比高。

Thread Pools

Starting a new thread for each task could limit throughput and cause poor performance. A thread pool is ideal to manage the number of tasks executing concurrently. JDK 1.5 uses the Executor interface for executing tasks in a thread pool and the ExecutorService interface for managing and controlling tasks. ExecutorService is a subinterface of Executor.



Creating Executors

To create an Executor object, use the **static** methods in the Executors class.

java.util.concurrent.Executors
+newFixedThreadPool(numberOfThreads: int): ExecutorService
+newCachedThreadPool(): ExecutorService

Creates a thread pool with a fixed number of threads executing concurrently. A thread may be reused to execute another task after its current task is finished.

Creates a thread pool that creates new threads as needed, but will reuse previously constructed threads when they are available.

[ExecutorDemo](#)

Run

```
2
3 public class ExecutorDemo {
4     public static void main(String[] args) {
5         // Create a fixed thread pool with maximum three threads
6         ExecutorService executor = Executors.newFixedThreadPool(3);
7
8         // Submit runnable tasks to the executor
9         executor.execute(new PrintChar('a', 100));
10        executor.execute(new PrintChar('b', 100));
11        executor.execute(new PrintNum(100));
12
13        // Shut down the executor
14        executor.shutdown();
15    }
16 }
17
```

SwingWorker

As discussed in § 29.6, all Swing GUI events are processed in a single *event dispatch thread*.

If an event requires a long time to process, the thread cannot attend to other tasks in the queue.

To solve this problem, you should run the time-consuming task for processing the event in a separate thread. Java 6 introduced SwingWorker.

SwingWorker可以帮助我们后台执行耗时的任务，而避免阻塞我们的应用程序，以让用户感觉不爽。

SwingWorker is an abstract class that implements Runnable. You can define a task class that extends SwingWorker, run the time-consuming task in the task, and update the GUI using the results produced from the task. Figure 29.28 defines SwingWorker.

SwingWorker

«interface»
java.lang.Runnable



javax.swing.SwingWorker<T, V>

#doInBackground(): T

#done(): void

+execute(): void

+get(): T

+isDone(): boolean

+cancel(): boolean

#publish(data V...): void

#process(data: java.util.List<V>): void

#setProgress(int progress): void

#getProgress(): void

Perform

Execute

finish

Schedul

Waits if

its res

Returns true if this task is completed.

Attempts to cancel this task.

Sends data for processing by the process method. This method is to be used from inside doInBackground to deliver intermediate results for processing on the event dispatch thread inside the process method. Note that V... denotes variant arguments.

Receives data from the publish method asynchronously on the Event Dispatch Thread.

Sets the progress bound property. The value should be from 0 to 100.

Returns the progress bound property.

SwingWorker有2个参数T, V

T: 为最终结果集

【<T> the result type returned by this SwingWorker's doInBackground and get methods】，由文档的介绍可以知道这个结果可以被doInBackground和get方法返回。

V: 为中间结果集

【<V> the type used for carrying out intermediate results by this SwingWorker's publish and process methods】

doInBackground是工作线程，他可以明确调用publish方法（注意publish方法只在SwingWorker类中实现），以发送中间结果V，然后这个中间结果有被发送到在EDT（事件派发线程）中的process方法中进行处理。当doInBackground处理完后，会自动调用done方法，由T类型的描述那里可以知道，在这个方法中可以调用get方法获取最终结果集。

#doInBackground():T //执行任务并返回T类型的结果

#done():void //结束doInBackground之后执行事件分发线程

+execute():void //安排这个SwingWorker来执行工作线程

+get():T //如果有必要则等待计算完成，然后获取它的结果（即doInBackground返回的结果）

+isDone():boolean //如果任务完成则返回true

+cancle():boolean //取消任务

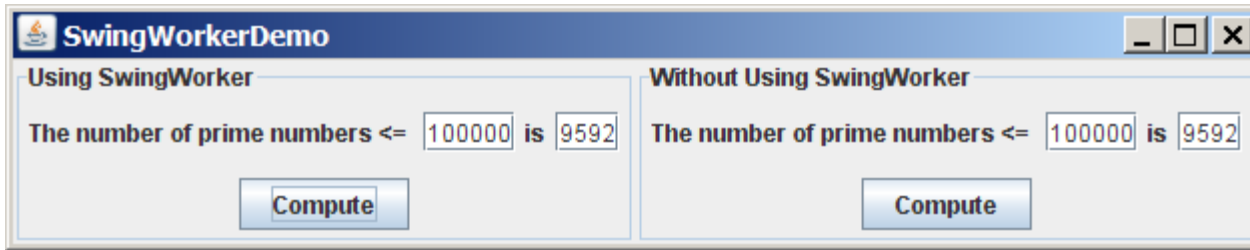
#publish(data v...):void //发送process方法要处理的数据（doInBackground中调用）。这个方法用于从doInBackground中传送中间结果， //以处理process方法中的事件分发线程。注意，v...表示变种参数

#process(data:java.util.List<V>):void //异步地接受事件分发线程上来自publish方法的数据

#setProgress(progress:int):void //设置进展约束属性。这个值应该从0到100

#getProgress():void //返回进展约束属性

SwingWorker Demo



SwingWorkerDemo

Run

```

jbtComputeWithSwingWorker.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        new ComputePrime(Integer.parseInt(jtfLimit1.getText()),
            jtfResult1).execute(); // Execute SwingWorker
    }
});

```

/** Task class for SwingWorker */

```

static class ComputePrime extends SwingWorker<Integer, Object> {
    private int limit;
    private JTextField result; // Text field in the UI

```

/** Construct a runnable Task */

```

public ComputePrime(int limit, JTextField result) {
    this.limit = limit;
    this.result = result;
}

```

/** Code run on a background thread */

```

protected Integer doInBackground() {
    return getNumberOfPrimes(limit);
}

```

/** Code executed after the background thread finishes

```

protected void done() {
    try {
        result.setText(get().toString()); // Display
    }
    catch (Exception ex) {
        result.setText(ex.getMessage());
    }
}

```

/** Return the number of primes <= limit */

```

public static int getNumberOfPrimes(int limit) {
    int count = 0; // Count the number of prime numbers
    int number = 2; // A number to be tested for primeness

```

// Repeatedly find prime numbers

```

while (number <= limit) {
    // Print the prime number and increase the count
    if (isPrime(number)) {
        count++; // Increase the count
    }

```

```

    // Check if the next number is prime
    number++;
}

```

```

return count;
}

```

/** Check whether number is prime */

```

private static boolean isPrime(int number) {
    for (int divisor = 2; divisor <= number / 2; divisor++) {
        if (number % divisor == 0) { // If true, number is not prime
            return false; // number is not a prime
        }
    }

```

```

return true; // number is prime
}

```

```

}

```

TIP

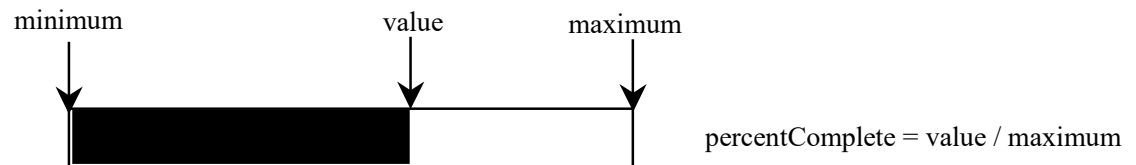
Two things to remember when writing Swing GUI programs,

- Time-consuming tasks should be run in SwingWorker.
- Swing components should be accessed from the event dispatch thread only.

JProgressBar

JProgressBar is a component that displays a value graphically within a bounded interval. A progress bar is typically used to show the percentage of completion of a lengthy operation; it comprises a rectangular bar that is "filled in" from left to right horizontally or from bottom to top vertically as the operation is performed. It provides the user with feedback on the progress of the operation. For example, when a file is being read, it alerts the user to the progress of the operation, thereby keeping the user attentive.

JProgressBar is often implemented using a thread to monitor the completion status of other threads. The progress bar can be displayed horizontally or vertically, as determined by its orientation property. The minimum, value, and maximum properties determine the minimum, current, and maximum length on the progress bar, as shown in Figure 9.20.



JProgressBar Methods

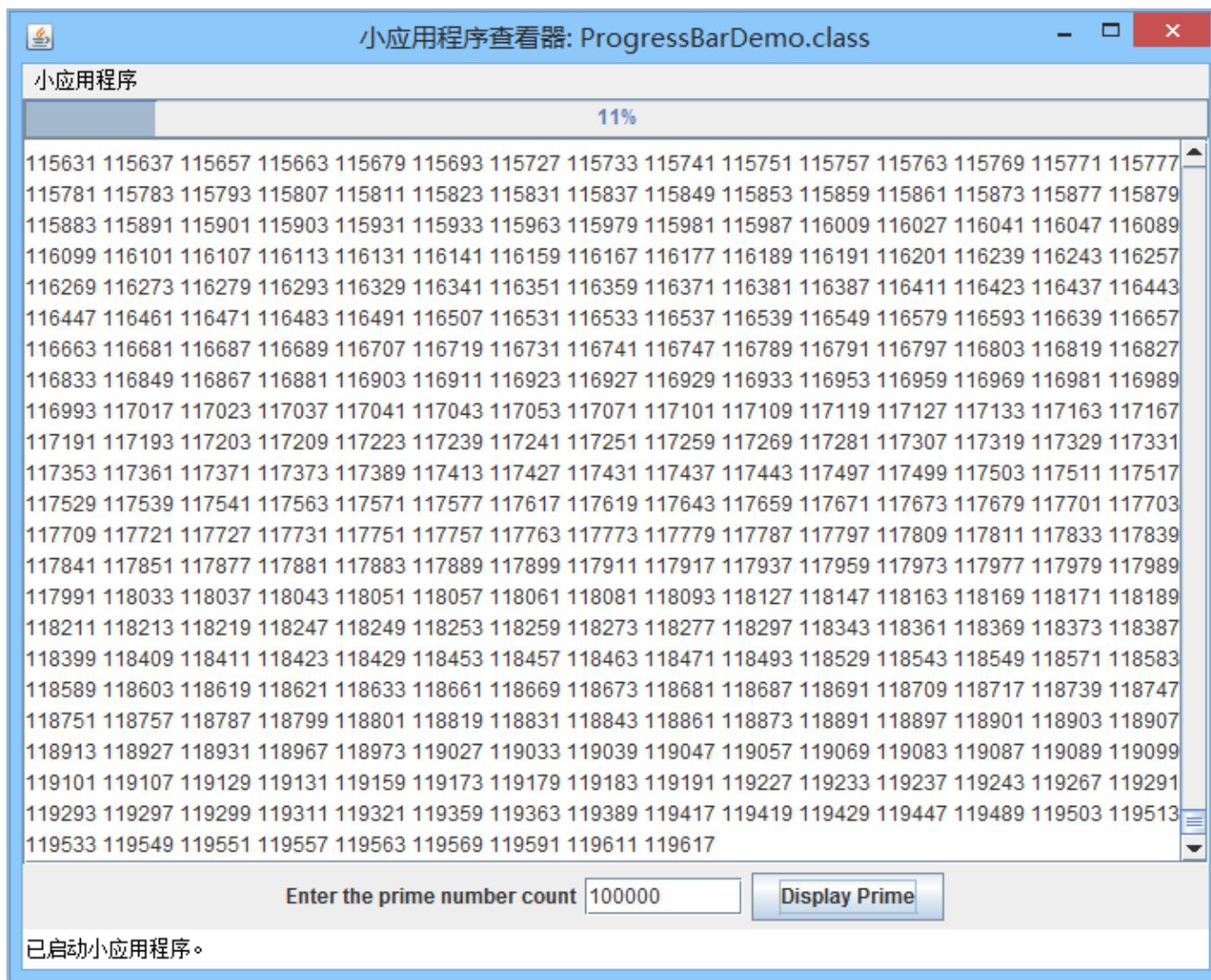
javax.swing.JComponent



javax.swing.JProgressBar

+JProgressBar()	Creates a horizontal progress bar with min 0 and max 100.
+JProgressBar(min: int, max: int)	Creates a horizontal progress bar with specified min and max.
+JProgressBar(orient: int)	Creates a progress bar with min 0 and max 100 and a specified orientation.
+JProgressBar(orient: int, min: int, max: int)	Creates a progress bar with a specified orientation, min, and max.
+getMaximum(): int	Gets the maximum value. (default: 100)
+setMaximum(n: int): void	Sets a new maximum value.
+getMinimum(): int	Gets the minimum value. (default: 0)
+setMinimum(n: int): void	Sets a new minimum value.
+getOrientation(): int	Gets the orientation value. (default: HORIZONTAL)
+setOrientation(orient: int): void	Sets a new minimum value.
+getPercentComplete():double	Returns the percent complete for the progress bar. 0 <= a value <= 1.0.
+getValue(): int	Returns the progress bar's current value
+setValue(n: int): void	Sets the progress bar's current value.
+getString(): String	Returns the current value of the progress string.
+setString(s: String): void	Sets the value of the progress string.
+isStringPainted(): Boolean	Returns the value of the stringPainted property.
+setStringPainted(b: boolean): void	Sets the value of the stringPainted property, which determines whether the progress bar should render a progress percentage string. (default: false)

SwingWorker和JProgressBar的配合使用



Example: ProgressBarDemo

SwingWorker和JProgressBar的配合使用

```
/** Task class for SwingWorker */
static class ComputePrime extends SwingWorker<Integer, Integer> {
    private int count;
    private JTextArea result; // Textarea in the UI

    /** Construct a runnable Task */
    public ComputePrime(int count, JTextArea result) {
        this.count = count;
        this.result = result;
    }

    /** Code run on a background thread */
    protected Integer doInBackground() {
        publishPrimeNumbers(count);

        return 0; // doInBackground must return a value
    }

    /** Override process to display published prime values */
    protected void process(java.util.List<Integer> list) {
        for (int i = 0; i < list.size(); i++)
            result.append(list.get(i) + " ");
    }

    /** Publish the first n primes number */
    private void publishPrimeNumbers(int n) {
        int count = 0; // Count the number of prime numbers
        int number = 2; // A number to be tested for primeness

        // Repeatedly find prime numbers
        while (count <= n) {
            // Print the prime number and increase the count
            if (isPrime(number)) {
                count++; // Increase the count
                setProgress(100 * count / n); // Update progress
                publish(number); // Publish the prime number
            }

            // Check if the next number is prime
            number++;
        }
    }
}
```

中间结果
类型

最终结果
类型

将list加入到result
这个JTextArea中

设置进度;
将number异步发
送给process函数

```
private JProgressBar jpb = new JProgressBar();

jpb.setStringPainted(true); // Paint the percent in a string
jpb.setValue(0);
jpb.setMaximum(100);

jbtDisplayPrime.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        ComputePrime task = new ComputePrime(
            Integer.parseInt(jtfPrimeCount.getText()), jtaResult);

        task.addPropertyChangeListener(new PropertyChangeListener() {
            public void propertyChange(PropertyChangeEvent e) {
                if ("progress".equals(e.getPropertyName())) {
                    jpb.setValue((Integer)e.getNewValue());
                }
            }
        });

        task.execute(); // Execute SwingWorker
    }
});
```

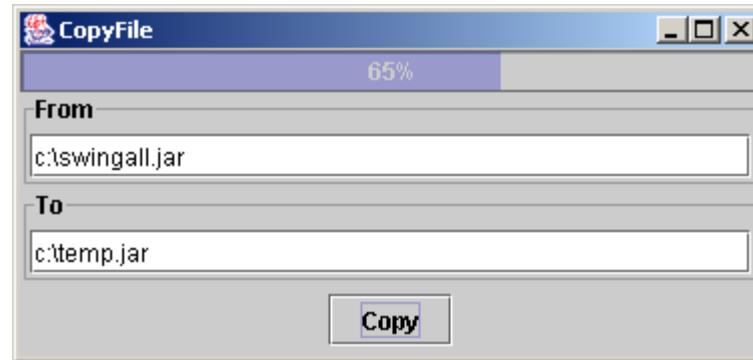
判断改变的属性是否是进度，如果是则获取进度的值并显示在进度条上

当从任务线程调用publish方法时，SwingWorker类调度process方法，如果SwingWorker通过publish发布了一些数据，那么也应该实现process方法来处理这些中间结果，任务对象的父类会在事件分发线程上激活process方法，因此在此方法中程序可以安全的更新UI组件。

无论何时调用setProgress方法，SwingWorker类都会产生一个propertyChangeEvent。setProgress方法设置一个0到100之间的新的进度值，这个值包装在PropertyChangeEvent中，这个事件的监听器使用getNewValue()方法获取进度值。

Example: JProgressBar Demo

Objective: Write a GUI application that lets you copy files. A progress bar is used to show the progress of the copying operation.



[CopyFile](#)

Run

```

class CopyFileTask implements Runnable {
    private int currentValue;

    public void run() {
        BufferedInputStream in = null;
        BufferedOutputStream out = null;
        try {
            // Create file input stream
            File inFile = new File(jtfFrom.getText().trim());
            in = new BufferedInputStream(new FileInputStream(inFile));

            // Create file output stream
            File outFile = new File(jtfTo.getText());
            out = new BufferedOutputStream(new FileOutputStream(outFile));

            // Get total bytes in the file
            long totalBytes = in.available();

            // Start progress meter bar
            jpb.setValue(0);
            jpb.setMaximum(100);

            int r;
            long bytesRead = 0;
            // You may increase buffer size to improve IO speed
            byte[] b = new byte[10];
            while ((r = in.read(b, 0, b.length)) != -1) {
                out.write(b, 0, r);
                bytesRead += r;
                currentValue = (int) (bytesRead * 100 / totalBytes);

                // Update the progress bar
                jpb.setValue(currentValue);
            }
        } catch (FileNotFoundException ex) {
            ex.printStackTrace();
        } catch (IOException ex) {
            ex.printStackTrace();
        } finally {
            try {

```

并发包

- Java有一套易用且高性能的并发开发工具
- java.util.concurrent下

```
public class Counter {  
    private int count;  
    private Object lock = new Object();  
    public void incr() {  
        synchronized(lock) {  
            count++;  
        }  
    }  
    public int getCount() {  
        synchronized(lock) {  
            return count;  
        }  
    }  
}
```

代价太大

用原子变量替代

原子变量

Atomic*	基本类型原子类	AtomicInteger AtomicLong AtomicBoolean
Atomic*Array	数组类型原子类	AtomicIntegerArray AtomicLongArray AtomicReferenceArray
Atomic*Reference	应用类型原子类	AtomicReference AtomicStampedReference AtomicMarkableReference
Atomic*FieldUpdater	升级类型原子类	AtomicIntegerFieldUpdater AtomicLongFieldUpdater AtomicReferenceFieldUpdater
*Adder	累加器	LongAdder DoubleAdder
*Accumulator	累加器	LongAccumulator DoubleAccumulator

AtomicInteger

```
private volatile int value;
```

```
/**  
 * Creates a new AtomicInteger with the given initial value.  
 *  
 * @param initialValue the initial value  
 */
```

```
public AtomicInteger(int initialValue) {  
    value = initialValue;  
}
```

```
/**  
 * Creates a new AtomicInteger with initial value {@code 0}.  
 */
```

```
public AtomicInteger() {  
}
```

包含一些以原子方式实现组合操作的方法：

int	accumulateAndGet (int x, IntBinaryOperator accumulatorFunction) Atomically updates the current value with the results of applying the given function to the current and given values, returning the updated value.
int	addAndGet (int delta) Atomically adds the given value to the current value.
boolean	compareAndSet (int expect, int update) Atomically sets the value to the given updated value if the current value == the expected value.
int	decrementAndGet () Atomically decrements by one the current value.
int	getAndAccumulate (int x, IntBinaryOperator accumulatorFunction) Atomically updates the current value with the results of applying the given function to the current and given values, returning the previous value.
int	getAndAdd (int delta) Atomically adds the given value to the current value.
int	getAndDecrement () Atomically decrements by one the current value.
int	getAndIncrement () Atomically increments by one the current value.
int	getAndSet (int newValue) Atomically sets to the given value and returns the old value.
int	getAndUpdate (IntUnaryOperator updateFunction) Atomically updates the current value with the results of applying the given function, returning the previous value.
int	incrementAndGet () Atomically increments by one the current value.

```
public final boolean compareAndSet(int expect, int update) {  
    return unsafe.compareAndSwapInt(this, valueOffset, expect, update);  
}
```

- `compareAndSet`是很重要的方法，比较并设置，简称为CAS。
 - 如果当前值等于`expect`，则更新为`update`；否则不更新；
 - 更新成功返回`true`，否则返回`false`

```
public final int incrementAndGet() {  
    for (;;) {  
        int current = get();  
        int next = current + 1;  
        if (compareAndSet(current, next))  
            return next;  
    }  
}
```

- `incrementAndGet()`方法在一个无限循环体内，不断尝试将一个比当前值大1的新值赋给自己，如果失败则说明在执行“获取-设置”操作的时已经被其它线程修改过了，于是便再次进入循环下一次操作，直到成功为止。这个便是`AtomicInteger`原子性的“诀窍”。

- 这种原子更新代表于synchronized锁不同的思维方式
 - Synchronized假设很可能冲突，所以需要先获得锁
 - CAS则是乐观的，认为冲突很少，只是在更新的时候才冲突检测
 - Synchronized代表阻塞式算法
 - CAS则是非阻塞式的，当冲突时，则重试，不会阻塞也不会有上下文切换开销。
- 非阻塞式容器
 - ConcurrentLinkedQueue, ConcurrentLinedDeque
 - ConcurrentSkipListMap, ConcurrentSkipListSet

- ReentrantLock底层利用AQS (AbstractQueuedSynchronizer)和CAS来实现。

```
public ReentrantLock(boolean fair) {  
    sync = fair ? new FairSync() : new NonfairSync();  
}  
  
public void lock() {  
    sync.lock();  
}  
  
static final class NonfairSync extends Sync {  
    private static final long serialVersionUID = 7316153563782823691L;  
  
    /**  
     * Performs lock. Try immediate barge, backing up to normal  
     * acquire on failure.  
     */  
    final void lock() {  
        if (compareAndSetState(0, 1))  
            setExclusiveOwnerThread(Thread.currentThread());  
        else  
            acquire(1);  
    }  
  
    protected final boolean tryAcquire(int acquires) {  
        return nonfairTryAcquire(acquires);  
    }  
}
```

如何把普通变量升级为原子变量？主要是 **AtomicIntegerFieldUpdater<T>**类

```
static class Goods {  
    //商品定价  
    volatile int price;  
}
```

```
static Goods phone;  
static Goods computer;  
  
AtomicIntegerFieldUpdater<Goods> atomicIntegerFieldUpdater =  
    AtomicIntegerFieldUpdater.newUpdater(Goods.class, "price");  
  
@Override  
public void run() {  
    for (int i = 0; i < 10000; i++) {  
        phone.price++;  
        atomicIntegerFieldUpdater.getAndIncrement(computer);  
    }  
}
```

并发容器

- java.util包中的大部分容器都是非线程安全的，若要在多线程中使用容器，你可以使用Collections提供的包装函数：synchronizedXXX，将普通容器变成线程安全的容器。但该方法仅仅是简单地给容器使用同步，效率很低。
- 因此并发大师Doug Lea提供了java.util.concurrent包，提供高效的并发容器。并且为了保持与普通的容器的接口一致性，仍然使用util包的接口，从而易于使用、易于理解。

- **同步容器Vector和HashTable**

- 早期的JDK在java.util包中提供了Vector和HashTable两个同步容器

```
//Vector
public synchronized int size() {};
public synchronized E get(int index) {};

//HashTable
public synchronized V put(K key, V value) {};
public synchronized V remove(Object key) {};
```

1. ConcurrentHashMap: 并发版 HashMap
2. CopyOnWriteArrayList: 并发版 ArrayList
3. CopyOnWriteArraySet: 并发 Set
4. ConcurrentLinkedQueue: 并发队列 (基于链表)
5. ConcurrentLinkedDeque: 并发队列 (基于双向链表)
6. ConcurrentSkipListMap: 基于跳表的并发 Map
7. ConcurrentSkipListSet: 基于跳表的并发 Set
8. ArrayBlockingQueue: 阻塞队列 (基于数组)
9. LinkedBlockingQueue: 阻塞队列 (基于链表)
10. LinkedBlockingDeque: 阻塞队列 (基于双向链表)
11. PriorityBlockingQueue: 线程安全的优先队列
12. SynchronousQueue: 读写成对的队列
13. LinkedTransferQueue: 基于链表的数据交换队列
14. DelayQueue: 延时队列

CopyOnWriteArrayList

- 并发版 ArrayList

- 底层结构也是数组，和 ArrayList 不同之处在于：当新增和删除元素时会创建一个新的数组，在新的数组中增加或者排除指定对象，最后用新增数组替换原来的数组。
- 适用场景：由于**读操作不加锁，写（增、删、改）操作加锁**，因此适用于读多写少的场景。
- 局限：由于读的时候不会加锁（读的效率 high，就和普通 ArrayList 一样），读取的当前副本，因此可能读取到脏数据。如果介意，建议不用。

//添加元素

```
public boolean add(E e) {  
    //独占锁  
    final ReentrantLock lock = this.lock;  
    lock.lock();  
    try {  
        Object[] elements = getArray();  
        int len = elements.length;  
        //复制一个新的数组newElements  
        Object[] newElements = Arrays.copyOf(elements, len + 1);  
        newElements[len] = e;  
        //修改后指向新的数组  
        setArray(newElements);  
        return true;  
    } finally {  
        lock.unlock();  
    }  
}  
  
public E get(int index) {  
    //未加锁, 直接获取  
    return get(getArray(), index);  
}
```

ConcurrentHashMap

- ConcurrentHashMap容器相较于CopyOnWrite容器在并发加锁粒度上有了更大一步的优化，它通过修改对单个hash桶元素加锁的达到了更细粒度的并发控制。(分段锁原理)

```
final V putVal(K key, V value, boolean onlyIfAbsent) {
    if (key == null || value == null) throw new NullPointerException();
    int hash = spread(key.hashCode()); //计算桶的hash值
    int binCount = 0;
    //循环插入元素，避免并发插入失败
    for (Node<K,V>[] tab = table;;) {
        Node<K,V> f; int n, i, fh;
        if (tab == null || (n = tab.length) == 0)
            tab = initTable();
        else if ((f = tabAt(tab, i = (n - 1) & hash)) == null) {
            //如果当前桶无元素，则通过cas操作插入新节点
            if (casTabAt(tab, i, null,
                new Node<K,V>(hash, key, value, null)))
                break;
        }
        //如果当前桶正在扩容，则协助扩容
        else if ((fh = f.hash) == MOVED)
            tab = helpTransfer(tab, f);
        else {
            V oldVal = null;
            //hash冲突时锁住当前需要添加节点的头元素，可能是链表头节点或者红黑树的根节点
            synchronized (f) {
                if (tabAt(tab, i) == f) {
                    if (fh >= 0) {
                        binCount = 1;
                        for (Node<K,V> e = f;; ++binCount) {
                            K ek;
                            if (e.hash == hash &&
                                ((ek = e.key) == key ||
                                 (ek != null && key.equals(ek)))) {
                                oldVal = e.val;
                                if (!onlyIfAbsent)
                                    e.val = value;
                                break;
                            }
                        }
                    }
                    Node<K,V> pred = e;
                    if ((e = e.next) == null) {
                        pred.next = new Node<K,V>(hash, key,
                                                value, null);
                        break;
                    }
                }
            }
        }
    }
}
```

ConcurrentSkipListMap

- List和Set有对应的CopyOnWriteArrayList与CopyOnWriteArraySet，HashMap有对应的ConcurrentHashMap，但是有序的TreeMap或并没有对应的ConcurrentTreeMap。
- 为什么没有ConcurrentTreeMap呢？这是因为TreeMap内部使用了红黑树来实现，红黑树是一种自平衡的二叉树，当树被修改时，需要重新平衡，重新平衡操作可能会影响树的大部分节点，如果并发量非常大的情况下，这就需要在许多树节点上添加互斥锁，那并发就失去了意义。
- 所以提供了另外一种并发下的有序map实现：
ConcurrentSkipListMap。

ConcurrentSkipListMap

- ConcurrentSkipListMap内部使用跳表（SkipList）这种数据结构来实现，他的结构相对红黑树来说非常简单理解，实现起来也相对简单，而且在理论上它的查找、插入、删除时间复杂度都为 $\log(n)$ 。在并发上，ConcurrentSkipListMap采用无锁的CAS+自旋来控制。

